Compression Enables Generalization: Wake-Sleep Cycles for Logic Programming with LLM Integration

Alexander Towell
Southern Illinois University Edwardsville
Edwardsville, IL, USA
lex@metafunctor.com
ORCID: 0000-0001-6443-9897

Hiroshi Fujinoki
Southern Illinois University Edwardsville
Edwardsville, IL, USA
hfujino@siue.edu

***Abstract*—In LLM-augmented rule learning, the operational question is no longer whether a system can discover rules but whether the discovered rules are genuine compression-driven generalizations or patterns the LLM memorized during training. We introduce a test protocol that makes this distinction auditable: fact bases over invented vocabulary the LLM cannot have seen in training, paired with a raw-LLM baseline on the same inputs. Under it, MDL-driven compression of an unannotated knowledge base recovers three rule types with a clean division of labor. Within-predicate generalizations come from symbolic compression: on a synthetic crafting domain it recovers 53% of held-out cases, an LLM-as-hypothesis-generator step raises this to 63%, and direct LLM prompting recovers 0%. Recursive transitive closures are the sharp case: a symbolic operation recovers them at 100% recall and 100% precision even on invented vocabulary where the raw LLM recovers 0%, so recursion, the rule type the LLM has most plainly memorized, is recovered without it. Cross-predicate rules are the one type symbolic compression cannot build from a fact base, so the LLM is the only route to them, even though a capable model can also induce the within-predicate and recursive rules symbolic compression already recovers for free. A capability ablation across a weak and a frontier model shows the LLM reaches the cross-predicate rules by structural induction rather than recall (the frontier model recovers one over fully invented predicate names, yet used as a raw inference engine recovers nothing), while its world knowledge is a double edge that can over-specify within-predicate cases. DreamLog, our system, runs wake-sleep cycles inspired by DreamCoder, with each sleep-phase compression verified against positive and negative queries under atomic rollback; on a canonical family tree the full pipeline reaches 80% recall at 100% precision, and it recovers the recursive ancestor closure that Popper, on this configuration, could not complete (its clingo backend ran unbounded past the timeout, so the target had to be excluded). Refining the length objective from clause count to a rename-invariant, parameter-free bits-based code makes the acceptance criterion structure-sensitive: it accepts an abstraction exactly when its reuse is deep enough to compress, and across a family of twenty-two synthetic domains the degree of compression predicts held-out recovery (Pearson $r = 0.52$, $p = 0.014$). These results are consistent with the Solomonoff thesis that shorter programs generalize better, while making the compression-versus-memorization distinction empirically testable.**

***Index Terms*—compression-based learning, logic programming, wake-sleep cycles, knowledge base compression, inductive logic programming, Solomonoff induction, LLM integration**

I. Introduction

Knowledge bases in logic programming accumulate facts over time, yet this accumulation does not constitute learning. A knowledge base with 300 ground facts about a family tree cannot determine whether a newly introduced person is someone's father, even though the concepts `father`, `mother`, and `grandparent` are implicit in the data. The facts grow, but understanding does not emerge.

This paper presents an empirical test of a classical theoretical claim: *compression is learning*. Solomonoff's theory of inductive inference [1] and the closely related Minimum Description Length (MDL) principle [2], [3] assert that the shortest program consistent with observations provides the best predictions. We test this claim in a logic programming setting, where "shorter program" means a smaller knowledge base (KB)—by clause count, or by a finer rename-invariant bits-based code we develop in Section III-C—and "better predictions" means the ability to derive facts about entities never seen during training.

We introduce DreamLog, a logic programming system that implements *wake-sleep cycles* inspired by DreamCoder [4]. During wake phases, DreamLog answers queries via SLD resolution, optionally invoking a large language model (LLM) to generate rules for undefined predicates. During sleep phases, nine compression operations transform accumulated facts into general rules. The operations are guided by a description-length objective and verified against a test suite of positive and negative queries, with atomic rollback ensuring behavioral preservation.

This paper makes two interlocked contributions: a test protocol that makes the compression-versus-memorization distinction auditable in LLM-augmented rule learners, and empirical evidence under that protocol that KB compression produces rules generalizing to unseen entities. We instantiate the protocol on two domains:

1) **A novel synthetic domain.** We construct a crafting system with 15 invented materials ("lumite," "vexal," "drossite") that no LLM has encountered

in training. Compression discovers concepts such as `master_artisan` and `safe_recipe` purely from structural patterns, achieving 53% recall on unseen entities through symbolic compression alone, and 63% with LLM-assisted operations (a raw LLM baseline achieves 0%).

2) **A canonical family tree.** On a 29-person, 4-generation family tree, the full pipeline achieves 80% recall on newly introduced individuals with 100% precision, compressing the KB from 300 to 184 clauses.

The novel domain result is the stronger validation. Because the LLM has never seen the invented terms, it cannot retrieve memorized rules from training data. The symbolic operations discover generalizations through MDL-driven compression of structural regularities in the data, without any labeled training examples or domain-specific inductive bias.

Our contributions are:

- **A test protocol** for auditing compression-driven generalization in LLM-assisted rule learners: a fact base over invented vocabulary plus a raw-LLM baseline on the same inputs, isolating compression-driven discovery from LLM training-data memorization.
- **A division-of-labor finding** that the protocol makes visible: symbolic MDL compression recovers within-predicate generalizations and recursive transitive closures (the latter at 100% on invented vocabulary where direct LLM prompting recovers 0%), while the LLM is the only route to cross-predicate rules, which symbolic compression cannot build from a fact base.
- **A capability ablation** (Section IV-D) showing the division of labor holds across model scale: a frontier model's cross-predicate contribution is structural induction, not recall (it recovers a cross-predicate rule over fully invented predicate names), with a world-knowledge bias that can over-specify within-predicate cases, while a weak local model contributes nothing and the system falls back entirely to the symbolic route.
- **Operation I**, a symbolic sleep operation that discovers recursive closures (e.g. `ancestor`) by detecting when one relation is the transitive closure of another, needing neither metarules nor labeled examples.
- **A bits-based description-length objective** (Section III-C): a rename-invariant, parameter-free refinement of clause count whose acceptance gate accepts an abstraction exactly when its reuse is deep enough to compress (Section IV-K). Under it, compression predicts held-out recovery across twenty-two domains, and its open-world variant lets recursive discovery recover partial closures the exact-match gate misses.
- **A head-to-head comparison** with Popper [5] (Section IV-F): comparable recall on the canonical domain but lower precision, lower recall on the novel-vocabulary domain, and the recursive `ancestor` closure that DreamLog recovers via Operation I, where Popper's clingo backend ran unbounded past the timeout and the target had to be excluded.
- **The wake-sleep architecture**: symbolic compression operations plus LLM-assisted rule discovery, all verified against behavioral test suites with atomic rollback.
- **An ablation and long-lived accumulation study** showing the complementary contributions of symbolic vs. LLM operations and an invest-then-compress-then-saturate dynamics consistent with DreamCoder's convergence.

II. Background and Related Work

A. Compression and Induction

Solomonoff's theory of inductive inference [1] formalizes the intuition that simpler explanations generalize better. The Kolmogorov complexity of a string is the length of the shortest program that produces it; Solomonoff induction predicts future data by weighting hypotheses inversely by their description length. While Kolmogorov complexity is uncomputable, the MDL principle [2], [3] provides a practical approximation: among models consistent with data, prefer the one with the shortest total description (model plus data given model). In our setting, the "model" is the set of rules in a KB, and the "data given model" is the set of remaining ground facts not derivable from those rules.

Modern empirical work supports the compression-prediction equivalence framing on which we rely. Delétang et al. [6] show that large language models are powerful general-purpose compressors and that the compression viewpoint clarifies scaling laws, tokenization, and in-context learning. Lotfi et al. [7] prove PAC-Bayes generalization bounds via parameter compression that are tight enough to explain why overparameterized neural networks generalize. We do not claim our MDL criterion is tight in the same sense (clause count is a coarse proxy for Kolmogorov complexity), but both lines of work ground the use of compression as a learning signal in measured rather than purely axiomatic terms.

B. MDL-Driven Compression of Logic Programs

The idea that compressing a logic program improves what it generalizes predates this work, and predates the LLM era, in two systems we must distinguish ourselves from carefully. Knorf [8] restructures an inductive logic programming knowledge base to minimize its size and redundancy, formulating refactoring as constraint optimization and introducing new predicates through invention during the rewrite. Its central empirical finding is exactly the thesis of this paper, established four years earlier: learning from a refactored (compressed) knowledge base improves an ILP system's predictive accuracy and reduces its learning time, because removing redundancy and factoring out shared structure leaves the knowledge in a more reusable form. Knorf's size-minimizing rewrite with predicate invention is the direct ancestor of our Operations A, B, and D, and its companion notion of *forgetting* background knowledge [9] parallels our usage-based pruning (Operation F). The Apperception Engine [10] makes the same point

from the unsupervised direction: it induces Datalog causal theories from raw sensory sequences under a parsimony (cost, i.e. description-length) objective, and is evaluated by how well those theories predict, retrodict, and impute *unseen* sensor readings, outperforming neural baselines. Apperception is the closest classical statement of our central claim that the shortest theory preserving the data generalizes best.

We therefore do not claim that MDL-driven recursion discovery is itself novel; both systems above already perform MDL-style induction of recursive, predicate-inventing logic programs, and our symbolic Operation I (which recovers a recursive transitive-closure rule such as `ancestor` from a flat fact base) sits squarely in their tradition. What neither system can do, because both predate widely available LLMs, is the contrast that isolates this mechanism from memorization. On recursion, the rule type a modern LLM has most plainly absorbed from its training corpus, we run the symbolic compressor and a raw LLM on the *same* task over a deliberately invented vocabulary that cannot appear in any pretraining set (Section IV). The compressor recovers the recursive rule and generalizes to unseen entities at 100%; the raw LLM, denied the lexical cues it would normally pattern-match, recovers 0%. The novel element is not the MDL machinery and not the LLM, but this division of labor made measurable: an invented-vocabulary protocol that attributes the recursion result to compression rather than recall, a distinction Knorf and Apperception had no occasion to draw.

C. DreamCoder and Library Learning

DreamCoder [4], [11] introduced wake-sleep library learning for program synthesis. During wake phases, it solves programming tasks using a library of primitives. During sleep phases, it compresses solved programs to extract reusable abstractions (via anti-unification of lambda terms), which become new library entries. Over iterations, the library converges to a small set of powerful primitives. DreamLog adapts this architecture to logic programming: our "library entries" are Horn clause rules, our "programs" are KB clauses, and our compression operations (particularly Operations C, D, and E) play the role of DreamCoder's abstraction phase.

Two successors to DreamCoder bear directly on our work. Stitch [12] reformulates the abstraction step as top-down corpus-guided synthesis, achieving 1000–10000$\times$ speedup over DreamCoder while preserving library quality. Operations D (predicate invention via skeleton fingerprinting) and E (body pattern extraction) play the role of Stitch for logic programs rather than λ-calculus programs: where Stitch identifies shared λ-term structure across solved tasks, our operations identify shared rule-body structure across the existing KB. LILO [13] extends DreamCoder with LLM-driven documentation and naming of discovered library functions, while keeping the symbolic compression mechanics. LILO is the closest published spirit to DreamLog: both compress a body of structured artifacts and use the LLM as a complement to symbolic discovery. The two systems differ in three ways. First, in what they compress: LILO's library entries are λ-calculus functions (e.g., `(λ x. cons x nil)` as a primitive); DreamLog's are Horn clauses (e.g., `father(X,Y) :- parent(X,Y), male(X)`). Second, in what the LLM contributes: LILO uses the LLM to name and document a library function whose body the symbolic engine has already discovered; DreamLog uses the LLM (Operation G) to *propose* a rule body that the symbolic engine could not have discovered (cross-predicate rules whose body literals span multiple functors, beyond the reach of within-predicate anti-unification). Third, in verification: LILO checks I/O correctness on held-out tasks; DreamLog checks query equivalence via the verification suite (Definition 1). LILO's input is a stream of *tasks with examples*; DreamLog's input is an *unannotated fact base*.

D. Inductive Logic Programming

Inductive Logic Programming (ILP) [14] learns logic programs from labeled positive and negative examples. FOIL [15] uses information gain to grow clause bodies. Progol [16] uses inverse entailment to construct the most specific hypothesis and searches for generalizations. Metagol [17] uses meta-interpretive learning with declarative bias in the form of metarules. ILASP [18] learns Answer Set Programs from partial interpretations. Popper [5] uses hypothesis testing and constraint learning to prune the search space. ∂ILP [19] and Neural Theorem Provers [20] differentiably learn rules from examples. LINC [21] combines LLMs with first-order logic provers for reasoning.

DreamLog differs from ILP in two key respects. First, DreamLog derives its training signal from the KB itself under the closed-world assumption, rather than requiring externally provided labeled examples. All ground facts serve as positive examples, and systematically generated perturbations serve as negative examples. Second, DreamLog's learning signal is compression, not classification accuracy. Rules are accepted only if they reduce description length while preserving the deductive closure of the KB.

E. Anti-Unification

Anti-unification, introduced by Plotkin [22] and Reynolds [23], computes the least general generalization (lgg) of two terms. It is the dual of Robinson's unification [24]: where unification finds the most general term subsuming both inputs, anti-unification finds the most specific term that both inputs are instances of. DreamLog uses anti-unification in Operations C and D to identify shared structure across groups of facts and rules.

F. Predicate Invention

Predicate invention [25], [26] creates new predicates not present in the background knowledge. Metagol [17] invents predicates using metarules. Predicate invention is considered one of the most challenging aspects of ILP [27]. DreamLog's Operation D discovers structurally identical rule sets via skeleton fingerprinting and extracts parameterized predicates using `call/N` dispatch, analogous to lambda abstraction and application.

G. Neural-Symbolic Integration

DeepProbLog [28] integrates neural networks with probabilistic logic programming. Logic Tensor Networks [29] ground logical formulas in real-valued tensors. NeuralLog and related approaches [30], [31] represent relations as learned embeddings. Knowledge graph completion methods (TransE [32], RotatE [33]) learn embedding-based link prediction.

A second cluster combines LLMs with symbolic solvers at inference time. Logic-LM [34] translates natural-language premises into a symbolic form, runs a classical solver, and refines via solver error messages; the LLM acts at parse time and the symbolic engine handles reasoning. LINC [21] follows the same pattern with first-order logic provers. DreamLog locates the LLM at a different stage: not premise translation but rule *learning*, where the LLM (Operation G) proposes candidate clauses that the symbolic evaluator verifies before they enter the KB. This preserves interpretability, since every derived fact retains a human-readable proof trace through Horn clause resolution, while exploiting LLM priors for the cross-predicate patterns that within-predicate anti-unification cannot reach.

Operation G belongs to a fast-growing family in which an LLM proposes hypotheses and a symbolic backend accepts or rejects them, and we position it against that family rather than claiming the pattern as new. The design rationale is established by two ICLR 2024 studies of inductive reasoning. Hypothesis Search [35] prompts an LLM for abstract hypotheses, realizes them as executable programs, and selects by execution on examples; Hypothesis Refinement [36] iterates propose-select-refine and finds that LLMs are excellent hypothesis *proposers* but unreliable rule *appliers*, so that pairing them with a symbolic interpreter that filters the proposed rules is what yields strong, generalizing results. This proposer-weak-but-verifiable-strong finding is precisely the premise of Operation G: the LLM supplies cross-predicate candidates and the SLD engine, not the LLM, decides what is sound. Two systems apply this premise to logic-program induction specifically. HtT [37] has an LLM induce and verify a reusable rule library and then deduce with it, controlling for memorization through base-randomized arithmetic and held-out CLUTRR kinship splits. The Idiap system [38] couples LLM theory refinement with a formal ILP engine and grades difficulty by rule-dependency structure, benchmarking head-to-head against Popper. Two 2025 systems extend the pattern further: ILP-CoT [39] has a multimodal LLM propose structurally correct rule skeletons that prune an ILP solver's search space, and Yang et al. [40] have multi-agent LLMs design the language bias (predicate vocabulary and type declarations) from raw text for an ILP solver to induce under. DreamLog differs from all six on three axes that we make explicit. First, input modality: every one of them consumes labeled examples, text-derived tasks, or an LLM-designed vocabulary, whereas Operation G consumes an unannotated, accumulating symbolic fact base and synthesizes its own positive and negative queries under the closed-world assumption. The closest published analog to that self-supervision is Poker [41], a meta-interpretive learner that automatically generates and labels its own positive and negative examples; but Poker is purely symbolic (no LLM route to cross-predicate rules), still requires at least one labeled positive per target and a second-order metarule bias, and carries no compression objective. Second, the acceptance signal: they retain rules by answer correctness, firing frequency, or F1 against examples, whereas Operation G retains a clause only if it shortens the KB's description length while preserving deductive closure, with a closed-world false-positive check and atomic rollback. Third, the verifier: it is a classical SLD resolution engine enforcing query equivalence, not the LLM itself and not an example-graded fitness score. We make no claim of novelty for the propose-then-verify pattern as of 2026; for a current map of this neighborhood under abduction, deduction, and induction we refer the reader to a recent survey [42]. Nor do we present our invented-vocabulary protocol as the first memorization control in this setting: HtT's base-randomized arithmetic and held-out kinship splits, and counterfactual perturbation of logical-reasoning puzzles [43], already isolate learned rules from recall. Our contribution is the particular instrument, an invented *predicate* vocabulary over an unannotated, accumulating fact base paired with a raw-LLM baseline, and its use to attribute each recovered rule type to compression or to the LLM.

H. Never-Ending Learning

NELL [44] continuously learns beliefs from the web, accumulating an ontology over years. DreamLog shares the goal of continuous knowledge accumulation but operates on a different axis: rather than acquiring new facts from external sources, it discovers structure in existing facts through compression. The two approaches are complementary.

III. System Design

DreamLog is a logic programming system with S-expression syntax that operates in alternating wake and sleep phases. This section describes the architecture and the nine compression operations that constitute the sleep phase.

A. Wake Phase

During the wake phase, DreamLog accepts assertions and queries. Assertions add ground facts (e.g., `(parent alice bob)`) or user-defined rules to the knowledge base. Queries are answered via SLD resolution [45] with negation-as-failure (NAF) [46], supporting:

- **`not/1`**: Negation-as-failure with floundering guard.
- **`call/N`**: Meta-predicate for runtime predicate dispatch.
- **Usage tracking**: Each clause records how often it fires during resolution, providing frequency data for sleep-phase operations.

When a query references an undefined predicate, the system optionally invokes an LLM to propose rules defining that predicate. Generated rules undergo structural validation, optional LLM-as-judge verification, and are added to the KB only if they pass all checks. This mechanism transforms undefined

Algorithm 1 Dream Cycle

Require: Knowledge base K, verification flag v
$K_0 \leftarrow \text{copy}(K)$
$S \leftarrow \text{BuildVerificationSuite}(K)$ {Pos/Neg queries}
$K \leftarrow \text{OpF}(K)$ {Dead clause pruning (wake data)}
$K \leftarrow \text{OpA}(K)$ {Subsumption elimination}
$K \leftarrow \text{OpB}(K)$ {Redundant fact pruning}
$K \leftarrow \text{OpC}(K, S)$ {Fact generalization}
$S \leftarrow \text{ExtendSuite}(S, K)$ {Add rule-derived queries}
$K \leftarrow \text{OpD}(K, S)$ {Predicate invention}
$K \leftarrow \text{OpE}(K, S)$ {Body pattern extraction}
$K \leftarrow \text{OpI}(K, S)$ {Recursive closure discovery}
$K \leftarrow \text{OpG}(K, S)$ {LLM-assisted compression}
$K \leftarrow \text{OpB}(K)$ {Re-prune (post-LLM)}
$K \leftarrow \text{OpH}(K)$ {Lemma caching}
if v **and** $\neg\text{Verify}(K, S)$ **then**
  $K \leftarrow K_0$ {Atomic rollback}
end if
return K, $|K|/|K_0|$

predicates from errors into opportunities for knowledge acquisition.

B. Sleep Phase: The Dream Cycle

The sleep phase compresses the KB through nine operations (A–I), executed in sequence. Algorithm 1 outlines the overall procedure. Operations that modify the KB (C, D, E, G, I) perform per-candidate verification against the test suite, accepting each candidate only if it preserves behavioral equivalence. Each accepted candidate must also satisfy the description-length criterion of Section III-C. A final pipeline-level verification with atomic rollback provides an additional safety net.

Definition 1 (Verification Suite): Given a KB K, the verification suite $S = (S^+, S^-)$ consists of:

- $S^+ = \{t \mid \text{Fact}(t) \in K\}$: all ground facts.
- S^-: synthetic negative queries formed by substituting atoms into existing fact templates at positions where those atoms do not appear in K.

A compression passes verification if every $q \in S^+$ remains derivable and no $q \in S^-$ becomes derivable.

C. The Description-Length Objective

Every KB-modifying operation routes its candidates through a single gate: a proposal that removes a set of clauses R and adds a set A is accepted only if it both preserves the verification suite (Definition 1) and does not increase the knowledge base's description length, $\text{DL}(K \setminus R \cup A) \leq \text{DL}(K)$. The choice of DL is the system's inductive prior. We expose two.

The default, retained for exact reproducibility, is the *clause-count* length $\text{DL}_\text{c}(K) = |K|$: every clause costs one unit. This is the criterion under which all earlier results were obtained. It is a coarse proxy for Kolmogorov complexity: it cannot see that one clause is structurally heavier than another, so an abstraction that trades several clauses for one is always favored regardless of how much internal structure the replacement carries.

The refinement is a *bits-based* two-part code, $\text{DL}_\text{b}(K) = L(D) + \sum_{c \in K} L(c \mid D)$, that prices a KB the way a prefix code transmitting it would. The dictionary D holds the distinct functors (keyed by name and arity) and constants occurring in K. Its cost charges each *distinct symbol* only for its arity, using the Elias γ length of $arity + 1$: constants cost 1 bit, arity-1 and arity-2 functors 3 bits, arity-3 and arity-4 functors 5 bits. Each clause then costs a 4-bit structural header plus, for every symbol occurrence, a 2-bit type tag and a payload of $\log_2 |F|$, $\log_2 |C|$, or $\log_2 V_c$ bits for a functor, constant, or variable, where $|F|$ and $|C|$ are the dictionary sizes and V_c the number of distinct variables in the clause. The proposal delta is computed exactly: changing a dictionary size re-prices every occurrence of that symbol class in the KB, and removing a symbol's last occurrence credits its declaration back.

The decisive property of this code is *rename invariance*: because a symbol is charged only for its arity, never for the length of its name, the description length is invariant under any consistent renaming of functors and constants. This is the right invariant for a compression objective over a logic program, whose symbol names are arbitrary labels: an invented predicate named `ex0` and one named `exception_has_non_vegan_ingredient` carry the same information. An earlier formulation that charged a symbol by the byte length of its name violated this invariant and was pathological in practice—a verbose auto-generated name could cost hundreds of bits and swamp the structural savings of the very abstraction that introduced it, so almost no abstraction ever paid. The arity code removes that artifact and is parameter-free.

The two objectives agree on operations that purely remove clauses (subsumption, redundant-fact pruning) and on recursive closure discovery, all of which strictly lower both lengths. They diverge on the abstraction operations—generalization, predicate invention, and body-pattern extraction—which add a definition and rewrite its uses. Clause count accepts such an abstraction whenever it nets even one fewer clause; the bits code accepts it only when the symbol occurrences it removes outweigh the occurrences and the declaration its definition adds. Section IV-K shows that this difference implements a principled threshold on when an abstraction is mature enough to compress.

D. Operation A: Subsumption Elimination

Removes clauses subsumed by more general clauses already in the KB. A rule R_1 subsumes R_2 if there exists a substitution θ such that $R_1\theta \sqsubseteq R_2$ and both have the same body length. Additionally, bodyless rules (rules with empty bodies) subsume matching facts. This removes redundancy introduced by prior operations or user input.

E. Operation B: Redundant Fact Pruning

Removes facts that are derivable from the remaining KB (rules plus other facts). Each fact f is tentatively removed;

if f remains derivable via SLD resolution, the removal is permanent. Batch removal is attempted first, with one-at-a-time fallback for mutually dependent facts. This is the primary mechanism by which rule discovery translates into fact removal.

F. Operation C: Fact Generalization with Exceptions

This is the central symbolic compression operation. Given a group of $n \geq 3$ facts sharing a functor and arity, the operation:

1) **Partitions** facts by argument position: for each position p, groups facts that agree on all arguments except position p.
2) **Finds a guard predicate**: a unary predicate g whose extension covers all values appearing at position p in the group (and possibly more).
3) **Computes exceptions**: values in the guard's extension not appearing in the fact group.
4) **Applies MDL**: the generalization is accepted only if $1 + |\text{exceptions}| < n$ (one rule plus exception facts is shorter than the original n facts).

Example 2: Given facts `(master_artisan kael)`, `(master_artisan sera)`, ..., `(master_artisan zara)` covering 7 of 9 artisans, and the guard predicate `artisan/1` covering all 9, Operation C produces:

```
(master_artisan X)
:- (artisan X), (not
(exception_master_artisan_artisan
X))
(exception_master_artisan_artisan
thom)
(exception_master_artisan_artisan
fen)
```

This replaces 7 facts with 1 rule + 2 exception facts = 3 clauses, a net reduction of 4.

G. Operation D: Predicate Invention

Discovers structurally identical rule sets across different predicates and extracts a parameterized "invented" predicate. The procedure:

1) **Extract skeletons**: For each predicate's rule set, compute a *skeleton* that abstracts functor names into roles (SELF for recursive calls, PARAM_i for distinct body functors) while preserving rule structure, arity, and variable connectivity.
2) **Group by skeleton**: Predicates with identical skeletons are structurally equivalent.
3) **Build parameterized predicate**: Create an invented predicate with an additional `call/N` dispatch parameter, and replace each original predicate with a one-line wrapper delegating to the invented predicate.
4) **Apply MDL**: Accept only if $k + m < m \cdot k$, where k is the number of rules per predicate and m is the number of predicates in the group.

This is the logic programming analog of lambda abstraction: the skeleton captures shared computational structure, and `call/N` dispatch plays the role of function application.

H. Operation E: Body Pattern Extraction

Finds common contiguous sub-goal sequences across rule bodies and extracts them as named predicates. Interface variables (those appearing both inside and outside the subsequence) become the extracted predicate's arguments. This discovers shared computational fragments across rules, analogous to common subexpression elimination.

I. Operation F: Dead Clause Pruning

Removes clauses with zero usage after sufficient wake-phase queries. Requires both a minimum query count and that at least 50% of predicates have been exercised. User-provided seed facts are protected from pruning; only clauses added by prior dream cycles (lemmas, LLM-generated rules) are eligible for removal.

J. Operation G: LLM-Assisted Compression

The symbolic operations (A–F) cannot synthesize a cross-predicate rule (e.g., `father(X,Y) :- parent(X,Y), male(X)`) from a fact-only knowledge base: Operation C generalizes within a single functor, and Operations D and E recombine existing rule bodies, of which a fact-only KB has none. Operation G invokes an LLM to propose such rules.

The pipeline:

1) **Prompt construction**: Sample facts ensuring every predicate is represented (round-robin), include predicate fact counts to guide directionality (specific $\leftarrow$ general).
2) **Response parsing**: Parse JSON-formatted rule proposals, handling common LLM formatting errors.
3) **Cycle filtering**: Reject rules creating cross-functor cycles in the dependency graph via DFS. Self-recursive rules (depth-limited by the evaluator) are permitted.
4) **Helper separation**: Separate helper predicates (new predicates supporting `not/1` patterns) from main rules (deriving existing facts).
5) **Evaluation**: Each main rule must derive ≥ 2 existing facts.
6) **False-positive check**: Enumerate solutions and reject rules deriving ground terms absent from the KB.
7) **Combined verification**: Verify the full set of accepted rules against the verification suite with bounded evaluation to prevent combinatorial explosion.

K. Operation H: Lemma Caching

Adds frequently derived intermediate terms as ground facts for faster resolution. Wake-phase derivation tracking identifies terms computed repeatedly during SLD resolution. Caching these as facts converts multi-step derivations into direct lookups, yielding up to 23.6$\times$ query speedup (Section IV).

L. Operation I: Recursive Closure Discovery

The symbolic operations (A–F) cannot discover recursive rules, and Operation G's prompt does not invite self-reference, so a recursive relation such as `ancestor` is never compressed and its full extension stays stored as ground facts. This is the single largest untapped compression opportunity:

the `ancestor` extension is exactly the transitive closure of `parent`, so a two-clause recursive definition can replace the entire stored closure. Operation I is the symbolic route to that recursion.

For each ordered pair of binary predicates (R, B), the operation tests whether R's ground extension equals the irreflexive transitive closure of B's extension. The procedure:

1) **Collect extensions**: gather the set of ground argument pairs for every binary predicate (over atom-valued arguments).
2) **Match against closure**: for each candidate base B with at least `min_base_facts` pairs, compute $\mathrm{TC}(B)$ by graph reachability and test $\mathrm{ext}(R) = \mathrm{TC}(B)$.
3) **Synthesize the pair**: on a match, build the base rule `R(X,Y) :- B(X,Y)` and the right-recursive rule `R(X,Z) :- B(X,Y), R(Y,Z).`
4) **Verify and replace**: verify the pair against the verification suite with the bounded evaluator, then replace R's facts with the two rules.

The MDL gain is large and unambiguous: two clauses replace the N stored closure facts, so acceptance turns entirely on verification. Termination is handled by construction. Left-recursion is excluded as a safeguard, and the bounded evaluator caps total resolution calls, so a candidate that does not terminate or over-runs the budget surfaces as a verification *failure* (the closure facts simply stop being derivable) rather than an exception. Operation I runs in the symbolic phase, after Operation E and before Operation G, so the symbolic-only and full pipelines differ only by Operation G. It is gated by a flag and off by default, preserving the zero-drift behavior of the standard pipeline. This version handles the transitive closure of a single binary base relation and right-recursion only; closure path length is bounded by the evaluator's recursion-depth limit.

Example 3: Given `parent` facts forming a chain `((parent a b), (parent b c), (parent c d), ...)` and the materialized closure stored as `ancestor` facts, Operation I detects that the `ancestor` extension equals TC(`parent`) and emits:

```
(ancestor X Y) :- (parent X Y)
(ancestor X Z) :- (parent X Y),
(ancestor Y Z)
```

After verification accepts the pair, the `ancestor` facts become redundant (each is re-derivable through the recursion), and Operation B prunes every one of them.

M. Behavioral Preservation

All operations share a common verification protocol. Before any modification, the system:

1) Takes a snapshot of the KB.
2) Constructs (or extends) the verification suite.
3) Applies the proposed compression.
4) Verifies that all positive queries remain derivable and no negative query becomes derivable.
5) Rolls back to the snapshot if verification fails.

This ensures that the compressed KB is *behaviorally equivalent* to the original on all tested queries. Experiment EX02 confirmed zero semantic drift across 200 held-out queries over multiple dream cycles.

IV. Experiments

We organize the experiments by the rule type each mechanism recovers, the division of labor the test protocol makes visible: within-predicate generalizations from symbolic compression (Section IV-B), recursive transitive closures from a symbolic operation (Section IV-C), and cross-predicate rules from LLM-proposed, symbolically-verified hypotheses (Section IV-E); a capability ablation (Section IV-D) then crosses all three rule types with a weak and a frontier model to test whether the division of labor is a property of the rule types or of one model. A head-to-head with the modern ILP system Popper (Section IV-F) and a set of supporting studies (long-lived accumulation, ablation, scaling, noise tolerance, and query speedup) follow. The per-rule-type and Popper experiments use Anthropic Claude Haiku 4.5 for the LLM-assisted operations; the capability ablation additionally uses a local `qwen2.5:3b` (free) and Claude Sonnet 4.6. Total LLM cost is a few dollars across all experiments, dominated by the Sonnet ablation (about $1.6).

A. Experimental Setup

All experiments run on DreamLog's Python implementation with SLD resolution, NAF, and `call/N`. The LLM provider for Operation G is Anthropic Claude Haiku 4.5 ($0.25/M input tokens) via API. Each dream cycle invokes the LLM at most twice (once for rule proposal in Operation G, once for predicate naming). Verification suites are constructed automatically from KB state. All compression operations use default parameters: minimum group size 3, shared structure threshold 0.1, maximum 200 prompt facts.

B. EX25b: Generalization on a Novel Domain

1) Motivation: The canonical test for compression-driven generalization (a family tree) is problematic: the LLM has likely seen `father(X,Y) :- parent(X,Y), male(X)` thousands of times during training. We cannot distinguish compression-driven discovery from training data recall. This experiment uses a synthetic domain with invented terms the LLM has never encountered.

2) Domain: An alchemical crafting workshop with 15 materials having invented names (lumite, vexal, drossite, pyreth, quarl, noctite, aerine, sylphex, thalline, morven, zephore, ethylis, fenrik, glacine, bramith) and properties (phase, material class, hazard status). The domain includes 16 recipes combining materials into products, 9 artisans with skills, and 5 skill requirements. Total: 112 base facts and 61 derived facts across 6 predicates (`hazardous_recipe`, `safe_recipe`, `same_phase_recipe`, `metallic_alloy`, `can_craft`, `master_artisan`).

TABLE I
GENERALIZATION ON THE NOVEL CRAFTING DOMAIN (EX25B). 33 CHECKS (19 POSITIVE, 14 NEGATIVE) ON UNSEEN ENTITIES. THE FIVE LLM-CONDITION RUNS RETURNED IDENTICAL RESULTS UNDER HAIKU 4.5; THE COMMITTED RUN ARTIFACT RECORDS THEM.

Condition	Rules	Acc	Prec	Recall
No dream	0	42%	—	0%
Raw LLM	0	42%	—	0%
Symbolic	5	52%	59%	53%
Full	19	58%	63%	63%

3) Protocol: Four conditions: (1) *no dream*: baseline KB with no compression; (2) *symbolic*: dream cycle with Operations A–F only (no LLM); (3) *full pipeline*: dream cycle with all operations including LLM-assisted Operation G; (4) *raw LLM*: prompt Haiku directly with all KB facts and new-entity queries (no compression pipeline). After dreaming, 6 new materials, 8 new recipes, and 4 new artisans are introduced with base facts only. 33 checks (19 positive, 14 negative) test whether derived predicates are correctly inferred for the unseen entities. LLM-dependent conditions are run 5 times; we report mean $\pm$ standard deviation (under Haiku 4.5 the five runs returned identical recall, so the standard deviation is zero and we report point values).

4) Results: Table I shows the results.

The *raw LLM* baseline achieves 0% recall: Haiku answers "NO" to every query about invented terms, confirming that direct LLM prompting contributes nothing on novel domains. The compression pipeline is genuinely necessary.

Symbolic compression alone discovers 5 rules and achieves 53% recall. The key discoveries are:

- `master_artisan(X) :- artisan(X), not(exception...)`: generalizes from the pattern that 7 of 9 artisans have 2+ skills.
- `safe_recipe(X) :- product(X), not(exception...)`: generalizes from the pattern that most recipes use non-hazardous materials.

These are discovered by Operation C through MDL-driven fact generalization. No domain knowledge is used; the system finds that "most artisans are master artisans" and "most recipes are safe" purely from the statistical structure of the data.

The full pipeline achieves 63% recall with roughly 19 discovered rules. Operation G adds cross-predicate rules including `hazardous_recipe` (via material hazard properties) and concepts like `volatile_material` and `metallic_material`. All five runs returned identical recall: the cross-predicate rules Operation G proposes for this domain are stable under Haiku 4.5, and this figure matches the DreamLog full-pipeline crafting recall reported independently in the Popper comparison (Table V, EX26).

Seven false positives occur from `safe_recipe` and `same_phase_recipe` generalizations applying to exception cases (e.g., a liquid-solid mix classified as same-phase). This illustrates the inherent risk of inductive generalization.

The undiscovered predicate is `can_craft(Artisan, Product)`, which requires a 3-hop join (artisan $\rightarrow$ skill $\rightarrow$ requires_skill $\rightarrow$ recipe_type). This exceeds the current operations' reach.

C. EX27: Recursive Closure Discovery

1) Motivation: Recursive rules, and the transitive closure in particular, are the canonical hard case for inductive logic programming. Metagol requires hand-supplied metarules to admit recursion, and Popper requires labeled positive and negative examples plus a recursion-enabling bias. EX26 had to exclude the recursive `ancestor` target entirely: neither DreamLog (whose Operations A–H find patterns within and across flat predicates but not recursive definitions) nor Popper (which hung in its `clingo` backend past the Python-level timeout) recovered it. Recursion is therefore the single largest gap in the preceding experiments. It is also the rule type the LLM has most plainly memorized: `ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z)` appears verbatim across countless Prolog tutorials. This makes it the sharpest available test of whether DreamLog's discovery is compression-driven or merely LLM recall. We add a symbolic operation, Operation I, that detects when a binary predicate `R` is the exact transitive closure of another binary predicate `B` and synthesizes the right-recursive definition, and we ask whether that discovery survives the invented-vocabulary protocol of EX25b.

2) Domains: Two transitive-closure domains, one canonical and one invented, mirroring the EX25/EX25b pairing. The canonical domain is `ancestor` as the transitive closure of `parent` over real first names (`alice`, `bob`, `carol`, ...), the relation and vocabulary the LLM has most certainly seen in training. The invented domain is `flux_reaches` as the transitive closure of `flux_links` over invented node names (`qux`, `vor`, `zane`, `plix`, ...) that the LLM has not encountered as graph nodes. Each training graph is an acyclic forward chain over 8 nodes with a few extra forward edges; the 28 training derived facts are the *exact* closure of the 8 training base edges, so Operation I's exact-match gate can fire.

3) Protocol: We use the new-entity protocol of EX25/EX25b rather than a holdout split, because holdout would leave the training closure incomplete and the exact-match gate would never trigger. We dream on the full training closure, then introduce 4 new entities with base edges only (extending the forward order so the combined graph stays acyclic), and check whether the discovered rule derives their closure pairs. The 76 new-entity checks per domain (38 positive, 38 closed-world-balanced negative) test reachability for pairs that touch a new node. Four conditions, seed 42: (1) *no dream*: baseline KB with no compression; (2) *symbolic*: Operation I only, no LLM; (3) *full*: Operation I followed by a recursion-aware Operation G; (4) *raw LLM*: Haiku used directly as an inference engine, answering yes/no on each held-out query with the training facts in context. This raw-LLM condition is a deliberately minimal inference probe—single-shot, yes/no, no chain-of-thought

TABLE II
RECURSIVE CLOSURE DISCOVERY (EX27). NEW-ENTITY PROTOCOL, SEED 42, 76 CHECKS PER DOMAIN (38 POSITIVE, 38 NEGATIVE). SYMBOLIC OPERATION I ALONE REACHES 100% RECALL AND 100% PRECISION ON BOTH DOMAINS, INCLUDING THE INVENTED ONE WHERE THE RAW LLM RECOVERS NOTHING.

Domain	Condition	Recall	Prec	Acc	Rules
Flux	No dream	0%	—	50%	0
(invented)	Raw LLM	0%	—	50%	0
	Symbolic	100%	100%	100%	2
	Full	100%	100%	100%	2
Family	No dream	0%	—	50%	0
(ancestor)	Raw LLM	2.6%	100%	51%	0
	Symbolic	100%	100%	100%	2
	Full	100%	100%	100%	4

and no few-shot exemplars—so its 0% is a statement about direct inference under that protocol, not a claim about the ceiling of LLM *reasoning*; the frontier-model raw column of EX28 (Section IV-D) is the stronger control, and it too recovers 0% on the synthetic recursive and cross-predicate cells. Verification is closed-world with the bounded evaluator; path lengths stay well under the evaluator's recursion-depth ceiling.

4) Results: Table II shows the results.

Symbolic compression alone, with no LLM, recovers the recursive definition on both domains and classifies every new-entity check correctly: 100% recall and 100% precision. On the invented `flux` domain it discovers

- `(flux_reaches X Y) :- (flux_links X Y)` and
- `(flux_reaches X Z) :- (flux_links X Y), (flux_reaches Y Z),`

the base and right-recursive clauses of the transitive closure, and compresses the closure to a ratio of 0.278 (the 28 derived facts collapse into 2 rules). The canonical `ancestor` domain yields the structurally identical pair over `parent`.

The *raw LLM* baseline, by contrast, recovers essentially nothing: 0% recall (0 of 38) on the invented vocabulary and 2.6% recall (1 of 38) on real names. Even though recursion over `parent` is the rule type the LLM has most obviously memorized, using the model directly as an inference engine over held-out queries fails. This is the decisive contrast: because the raw LLM gets 0% on `flux_reaches`, the 100% symbolic result on that domain cannot be memorization. Operation I recovers an unbounded relation from its finite extension purely from MDL-driven structure, with no prior knowledge of the vocabulary.

We are explicit that the *full* pipeline adds nothing here. Operation I runs before Operation G, so by the time the LLM is consulted the recursive rule is already in place and the closure is already compressed; the LLM's recursion proposal is redundant on a clean closure. On `family` the recursion-aware Operation G proposed 2 additional rules (duplicating the clauses Operation I had already synthesized), and recall, precision, and the compression ratio were unchanged; on `flux` it proposed none. The *full* row therefore equals the *symbolic* row, and the LLM should not be credited with the recovery. The LLM cost across the raw-LLM and full conditions was $0.28 (154 Haiku calls), spent almost entirely on the raw-LLM baseline that failed.

5) Interpretation: This is the sharp instance of the division of labor that organizes this section. Recursive closures are a clean *symbolic* capability: Operation I recovers them on vocabulary the LLM never saw, while the LLM, used directly as an inference engine, recovers them on neither the familiar nor the unfamiliar domain. Read together with the cross-predicate experiments, where the LLM is essential because symbolic operations cannot cross functor boundaries, the picture is that the two mechanisms cover disjoint rule types. The result also updates the EX26 framing: where that comparison recorded that neither DreamLog nor Popper recovers `ancestor`, DreamLog now does, via a symbolic operation that needs no labels and no metarules. A capable LLM used as a hypothesis *proposer*, rather than as the inference engine measured here, can in fact induce the recursive rule too (Section IV-D); the point in this section is that it is not needed, because Operation I recovers the closure deterministically and at no LLM cost.

We state the boundary plainly. These results are on controlled synthetic closures: exact transitive closure, right-recursion only, and small acyclic graphs. Operation I's exact-match gate is closed-world, so a holdout-style partial closure (where some training closure pairs are withheld) does not trigger it here; the open-world subset relaxation that does recover such partial closures is developed and evaluated in Section IV-K (EX35–37). Operation I is off by default and flag-gated, preserving the zero-drift guarantee of the production pipeline.

D. EX28: The Division of Labor Across Model Capability

The preceding experiments fix the LLM (Haiku) and organize the evidence by rule type. EX28 holds the rule-type structure fixed in one controlled design and varies what those sections could not: model capability and a per-mechanism ablation that isolates each route. The question is whether the division of labor is a property of the rule types or an artifact of one model.

1) Design: A clean 3×2 grid crosses three rule types (within-predicate, recursive, cross-predicate) with two vocabularies (canonical, fully synthetic). The synthetic column invents the *predicate* names, not only the entities, so the only recoverable signal is structural: recursion becomes `tessik` as the closure of `vorlan`, and the cross-predicate rule becomes `brulit(X,Y) :- klanth(X,Y), dosq(X),` names with no English substring and no semantic hint. (We are deliberate about this: an earlier version used `flux_reaches`, but "reaches" leaks the reachability semantics of transitive closure; renaming to `tessik` dropped the raw-LLM inference score on that cell from 0.76 to 0.00 while leaving the rule-induction score unchanged at 1.00, confirming the lexical cue had aided pure inference but never induction.

Both the superseded leaky-vocabulary pilot run and the fully synthetic run are retained in the registry for provenance.)

Each cell runs four conditions: *symbolic-only* (the rule type's owning operation, no LLM), *LLM-only* (that owning operation switched off, Operation G on), *full* (both), and *raw-LLM* (the model used directly as an inference engine on held-out queries). Alongside recovery we report a *proposal rate*: over 30 independent dreams, how often Operation G proposes a rule structurally equivalent to the target, where structural equivalence is identity up to variable renaming and body-literal reordering under a single consistent variable bijection (so left- and right-recursion are distinguished). Two models anchor the capability axis: `qwen2.5:3b` (a small local model) and Claude Sonnet 4.6 (a capable frontier model). New-entity protocol, seed 42, 30 proposal probes per LLM cell. We note one asymmetry: the weak model's synthetic-column cells were collected on the earlier, mildly leaky vocabulary and not re-run after the rename to fully synthetic names. This does not affect the weak-model conclusion, whose proposal rate is 0.00 on every cell and every vocabulary; the capable-model grid uses the fully synthetic names throughout.

2) Results: Table III reports recovery for every cell and condition; proposal rates are given inline below.

a) Symbolic compression is vocabulary-independent; the weak LLM adds nothing.: Symbolic-only recovers within-predicate and recursive structure at 100% recall on both the canonical and the synthetic vocabulary, and recovers cross-predicate at 0% (no symbolic operation can build a cross-functor rule from a fact-only KB). This extends EX27 (recursion) to a second rule type and re-confirms vocabulary-independence. (Table III reports recall; within-predicate symbolic *precision* is 0.67 rather than 1.00, because the new-entity protocol introduces one exception individual without its exception fact, so Operation C's rule over-derives it. This is the cost of predicting an unseen exception, not a recovery failure, and the recursive cells stay at precision 1.00.) The weak model proposes zero structurally-correct rules in any cell (proposal rate 0.00 throughout): inspected, it echoes the derived facts back as a JSON list, proposes ground instances such as `father(tom,ann):-parent(tom,ann)`, or proposes the reversed direction `parent:-father,male`. Operation G's verified acceptance rejects all of these, so the full column equals the symbolic column for `qwen2.5:3b`, exactly the "full adds nothing" pattern EX27 reports for Haiku.

b) A capable LLM contributes cross-predicate rules by structural induction, not semantic recall.: Sonnet's LLM-only condition recovers cross-predicate at 100% on *both* vocabularies, with proposal rate 1.00 on canonical and 1.00 on synthetic; it also recovers recursion on both (proposal 1.00). The synthetic results are decisive: Sonnet proposed `brulit(X,Y):-klanth(X,Y),dosq(X)` and the full recursive pair over `tessik`/`vorlan`, exactly the targets, on predicates carrying no meaning. This is genuine structural induction from fact co-occurrence, not retrieval of a memorized rule. We read it as existence-proof evidence that the cross-predicate contribution *can* be structural, established on one capable model, one seed, and 30 proposal probes per cell over small synthetic domains, rather than as a scaling law. Crucially, the not-memorization argument of EX27 survives unchanged: the same Sonnet model used as a raw *inference engine* still recovers 0% on the synthetic recursive and cross cells (Table III, Raw column). The capability lives in the model as a hypothesis *proposer* filtered by the symbolic verifier, not in the model as a query answerer. With the capable model the full pipeline reaches 1.00 on all six cells: the LLM supplies exactly the cross-predicate rules that symbolic compression provably cannot build, and symbolic compression supplies the within-predicate and recursive rules deterministically and at zero LLM cost.

c) World knowledge is a double edge.: The one cell where Sonnet's LLM-only condition fails is within-predicate *canonical* (recovery 0.00, proposal 0.00), while it succeeds on within-predicate *synthetic* (recovery 1.00, proposal 0.53). The cause is instructive. On the canonical `can_fly` domain Sonnet proposes `can_fly(X):-bird(X),not(non_flying_bird(X))` together with `non_flying_bird(X):-bird(X),not(can_fly(X` it knows from the world that some birds are flightless, so it imports an exception predicate the data never mentioned. The resulting definition both misses the bare `can_fly:-bird` generalization the extension supports and is circular (unstratified negation), which the SLD verifier rejects. On the meaningless `glonk`/`zorp` domain Sonnet has no such knowledge and proposes the clean rule. World-knowledge recall, then, both enables cross-predicate semantics and sabotages within-predicate generalization, and the invented-vocabulary protocol is what renders the two effects separately measurable.

3) Interpretation: EX28 sharpens the division of labor rather than dissolving it. The necessity structure holds across model scale: symbolic compression is sufficient, deterministic, verified, and free for the rule types it can build from extensional facts (within-predicate generalization and recursive closure) on any vocabulary, and the LLM is the only route to cross-predicate rules. What the capable-model anchor refines is the *character* of the LLM's contribution. It is not semantic recall of familiar rules: Sonnet induces the correct cross-predicate and recursive rules on predicates with no meaning, by structure alone. And it is not free of hazard: the same world knowledge that lets the model name a cross-predicate relation leads it, on canonical within-predicate cases, to over-specify with real-world exceptions the data does not ask for. The weak model is the floor (it proposes nothing usable, so a system built on it falls back entirely to the symbolic route); the capable model is the genuine neurosymbolic regime, in which the LLM is an inductive reasoner whose proposals the symbolic verifier still has the final word on. We make no claim that frontier capability is required for the symbolic results, which are model-independent; the claim is narrower and, we think, more useful: the protocol measures what each component contributes, and the contribution of the LLM is structural induction at the cross-predicate boundary, bounded

TABLE III

THE DIVISION OF LABOR ACROSS MODEL CAPABILITY (EX28). RECOVERY (RECALL) BY RULE TYPE × VOCABULARY, FOR SYMBOLIC-ONLY (VOCABULARY-INDEPENDENT, IDENTICAL FOR BOTH MODELS), AND FOR THE LLM-ONLY AND FULL CONDITIONS UNDER A WEAK MODEL (`QWEN2.5:3B`) AND A CAPABLE MODEL (SONNET 4.6). SYMBOLIC COMPRESSION RECOVERS WITHIN-PREDICATE AND RECURSIVE STRUCTURE ON ANY VOCABULARY BUT CANNOT BUILD CROSS-PREDICATE RULES; THE WEAK LLM ADDS NOTHING (FULL = SYMBOLIC); THE CAPABLE LLM RECOVERS CROSS-PREDICATE ON BOTH VOCABULARIES AND MAKES THE FULL PIPELINE COMPLETE (1.00 ON ALL SIX CELLS).

			qwen2.5:3b		Sonnet 4.6		
Rule type	**Vocab**	**Symbolic**	**LLM-only**	**Full**	**LLM-only**	**Full**	**Raw (q/S)**
Within	canonical	1.00	0.00	1.00	0.00	1.00	0.00 / 0.00
Within	synthetic	1.00	0.00	1.00	1.00	1.00	0.00 / 0.00
Recursive	canonical	1.00	0.00	1.00	1.00	1.00	0.00 / 1.00
Recursive	synthetic	1.00	0.00	1.00	1.00	1.00	0.00 / 0.00
Cross	canonical	0.00	0.00	0.00	1.00	1.00	0.00 / 1.00
Cross	synthetic	0.00	0.00	0.00	1.00	1.00	0.00 / 0.00

TABLE IV

GENERALIZATION ON THE CANONICAL FAMILY TREE (EX25). THE FULL PIPELINE ACHIEVES 80% RECALL WITH 100% PRECISION.

Condition	**Rules**	**Acc**	**Prec**	**Recall**	**KB Size**
No dream	0	29%	—	0%	300
Symbolic	1	29%	—	0%	—
Full	7	86%	100%	80%	184

by a verifier and shadowed by a world-knowledge bias that the invented vocabulary exposes.

E. EX25: Generalization on a Canonical Domain

1) Domain: A 4-generation family tree with 29 people, 96 base facts (`person`, `male`/`female`, `parent`), and 204 derived facts across 7 predicates (`father`, `mother`, `grandparent`, `grandfather`, `grandmother`, `great_grandparent`, `ancestor`).

2) Protocol: Same conditions as EX25b, including the raw LLM baseline. After dreaming, 6 new people are introduced with base facts only, and 14 checks (10 positive, 4 negative) test derived relationships.

3) Results: Table IV shows the results.

The full pipeline discovers 7 rules: `father` ← `parent` + `male`, `mother` ← `parent` + `female`, `grandparent` ← `parent` ∘ `parent`, `grandfather` ← `grandparent` + `male`, `grandmother` ← `grandparent` + `female`, and variants. The KB compresses from 300 to 184 clauses (ratio 0.613). Precision is 100%: no false positives.

The sole unrecovered predicate is `ancestor`, which requires a recursive rule (`ancestor(X,Y) :- parent(X,Y)` and `ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z)`). The current LLM prompt does not propose recursive rules, and the original symbolic operations (A–H) cannot discover them from flat fact patterns; the recursive case is taken up by Operation I (Section IV-C), which recovers `ancestor` as the closure of `parent`.

To check that the LLM's cross-predicate contribution is not an artifact of this one tree, we ran the full pipeline across six structurally varied family trees, holding out cross-predicate facts under the new-entity protocol and isolating the cross-predicate rules from the within-predicate and recursive ones (EX39). The contribution is robust in direction but variable in coverage: full-pipeline cross-predicate recall is $54.2 \pm 22.4\%$ against a 0% symbolic baseline, at 100% precision on every tree, with Haiku proposing `father` and `mother` reliably while the multi-hop `grandfather` and `grandmother` rules are recovered on some trees and not others—the source of the spread. The division of labor (the LLM is the only route to cross-predicate rules) holds at $n = 6$; its *completeness* on multi-hop relations does not, a caveat the single-domain figure hid. (This cross-predicate-only recall is a stricter slice than the 80% overall recall above, which counts within-predicate and cross-predicate recoveries together on the one tree.)

Symbolic compression discovers only one entity-specific rule (e.g., "albert is an ancestor of everyone") rather than the general concept. This highlights the critical role of the LLM: symbolic operations find patterns *within* a predicate, while the LLM discovers relationships *across* predicates.

F. EX26: Popper ILP Baseline

1) Motivation: DreamLog and Popper [5] occupy different points in the rule-learning design space. Popper is supervised (it requires positive and negative examples plus mode declarations) and uses a constraint-driven search; DreamLog is unsupervised (it consumes an unannotated fact base) and uses a compression-driven search. A naive head-to-head would advantage one system or the other depending on how labels are supplied. We construct an apples-to-apples comparison that gives each system the natural form of its required inputs and evaluates both on the same held-out facts. We are not the first to benchmark an LLM-assisted learner against Popper: the Idiap system [38] reports an LLM-theory-refinement-versus-Popper comparison graded by rule-dependency difficulty. EX26 differs in that both systems consume the same unannotated fact base rather than labeled example sets, and in pitting compression-driven against constraint-driven search on identical held-out facts.

2) Protocol: For each target predicate, the same EX25/EX25b fact base is converted to Popper input format: `bk.pl` contains the base facts plus the other targets'

TABLE V
EX26 POPPER BASELINE. APPLES-TO-APPLES COMPARISON ON THE SAME EX25 FAMILY AND EX25B CRAFTING DOMAINS, THREE SEEDS PER CELL. DREAMLOG CONDITIONS MATCH EX25/EX25B NUMBERS WITHIN ROUNDING; POPPER IS THE NEW DATA POINT.

Domain	System	Recall	Prec	Acc	Rules
Family	No dream	0%	—	29%	0
	Symbolic	0%	—	29%	1
	Full	80%	100%	86%	7
	Popper	80%	67–80%	57–71%	6
Crafting	No dream	0%	—	42%	0
	Symbolic	53%	59%	52%	5
	Full	63%	63%	58%	21–24
	Popper	42%	80%	61%	4

positives; `exs.pl` contains the held-in derived facts of the target predicate as E^+ and a closed-world random sample (with $|E^-| = |E^+|$, drawn with the same RNG seed as the data split) as E^-; `bias.pl` contains mode declarations from the natural schema. Mode declarations are emitted programmatically from a fixed template embedded in the experiment script, committed to version control before the first result row was recorded; no `bias.pl` is hand-edited after a result is observed. Per-cell timeout is 300 seconds; failures (timeout, parse error, crash) are recorded as zero rules. Recorded Popper runtimes were sub-second for every target on every seed (maximum 1.12 s), well below the budget. We exclude the recursive `ancestor` target: Popper's Python-level timeout does not interrupt its `clingo` C-extension on this configuration, leaving the call unbounded in practice, and EX25 also does not recover `ancestor`, so both systems get a pass on the recursive case here (we revisit recursion in Section IV-C, where Operation I recovers it). Three random seeds (42, 43, 44) per (system, domain) cell, yielding 24 cells total (4 systems $\times$ 2 domains $\times$ 3 seeds). Total wall time was approximately four minutes; LLM cost $0.03 across the six full-pipeline cells.

3) Results: Table V reports recall, precision, accuracy, and rule count for each cell, averaged over the three seeds. Per-seed values were within 1 percentage point of the cell mean for all reported conditions: Popper is deterministic given a fixed seed (negative samples and search order are seed-determined), and DreamLog full pipeline showed only minor rule-count variance (21–24 on crafting) with identical recall and precision across seeds. The $n = 3$ sample is too small for informative bootstrap confidence intervals, and the reported ranges reflect the full seed-to-seed spread. The 11-percentage-point gap between Popper and DreamLog symbolic on crafting holds at every individual seed; the per-seed Popper crafting recall is 42.4% on all three.

On the family domain, Popper matches DreamLog's full-pipeline recall (both 80%) but trails on precision by 20–33 percentage points. The rules tell the story: Popper discovers `father(X,Y) :- parent(X,Y)` and `mother(X,Y) :- parent(X,Y)` (and analogous over-general grandfather/grandmother rules) because random closed-world negatives do not include "parent but female" near-misses that would force the gender constraint. The two correct rules Popper discovers are `grandparent` and `great_grandparent`, where no auxiliary predicate is needed.

On the crafting domain, the result inverts. Popper's recall (42%) is *below* DreamLog's symbolic-only baseline (53%) by 11 percentage points, and well below the full pipeline (63%). Both Popper and DreamLog-symbolic are pure symbolic learners with no LLM, so the gap argues that DreamLog's MDL-guided fact generalization (Operation C) and skeleton-based abstraction (Operations D, E) have an inductive bias that random-negative ILP does not naturally express. Popper compensates with higher precision (80% vs. DreamLog full's 63%), producing four conservative rules rather than 20+ exploratory ones.

4) Interpretation: We do not read these numbers as "DreamLog beats Popper" or vice versa. The two systems answer different signals. The findings characterize the methodological tradeoff cleanly. On family, the closed-world random-negative protocol is too easy for Popper because the universe of false "father" tuples is dominated by clearly-impossible pairs (e.g., a child as parent of their grandparent); a curated set of near-miss negatives would close the gap. On crafting, hand-curating equivalent near-misses is not natural because the invented vocabulary does not come with a pre-existing intuition about what should be excluded. DreamLog's contribution is operational rather than competitive: when only an unannotated fact base is available, LLM-supplied semantic priors (Operation G) and MDL-guided structural generalization (Operations C, D, E) substitute for the curated supervision Popper relies on.

5) Two more ILP baselines: Metagol and ILASP (EX40–41): Popper is constraint-driven; we add a meta-interpretive learner, Metagol [17], as a second point of comparison, on the two rule types that most sharply distinguish the systems. Given the standard metarules—one conjunction template ($P(A,B) \leftarrow Q(A,B), R(A)$) for the cross-predicate case and the `ident` and `tailrec` templates for recursion—Metagol recovers both `father(X,Y) :- parent(X,Y), male(X)` and the right-recursive `ancestor` definition, each at 100% recall and 100% precision on the new-entity holdout, in tens of milliseconds. The result is clean, but the comparison turns on what had to be supplied. Metagol's recovery of `ancestor` is guaranteed once the `tailrec` metarule is given: the recursive definition is the only two-clause program consistent with that template and the examples, so the learner cannot fail. DreamLog's Operation I recovers the same rule with no template, from the fact extension alone, and on invented vocabulary the LLM never saw (Section IV-C); Popper, given no such template, hung on the recursive target.

ILASP [18], which learns Answer Set Programs from labeled examples under a mode-declaration language bias, gives a third reference point (EX41). Supplied with mode declarations and positive and negative examples, ILASP recovers

`father(X,Y) :- parent(X,Y), male(X)` in about a second and the recursive `ancestor` program at 100% recall and precision on the holdout. Unlike Popper, ILASP learns recursion natively; unlike Metagol, it does not fix the clause shape, so its search space is larger and recursion is slower—we reduced the family tree to a five-node chain for the `ancestor` run to terminate, where Operation I recovers the closure on the full twenty-nine-person tree. The three baselines arrange on a prior-knowledge spectrum: DreamLog learns from an unannotated fact base with neither labels nor a language bias; Popper and ILASP require labeled positive and negative examples plus a bias (learned constraints, mode declarations); Metagol additionally requires metarule templates that fix the clause shape. DreamLog sits at the low-supervision end and still recovers the cross-predicate rule (via Operation G) and the recursive closure (via Operation I) that the supervised systems recover only with that added prior knowledge—the operational claim of this work, now bracketed by three ILP paradigms rather than one.

G. EX25c: Holdout Sweep and the Open-World Mode

1) Motivation: The new-entity tests (EX25, EX25b) introduce entities with no prior facts at all. A complementary protocol, common in inductive logic programming, holds out a fraction of derived facts and asks whether the system can recover them after compressing the remainder. This sweep characterizes the data density threshold for compression-driven generalization on the family tree domain.

2) Closed-World versus Open-World Verification: Operation G's verification pipeline includes a false-positive check: any rule that derives a ground term absent from the KB is rejected. This is correct for the standard compression setting (a complete KB should not be augmented with novel facts) but pathological for holdout evaluation, where the goal is precisely to recover absent facts.

We introduce an *open-world mode* that disables this check. In this mode, Op G accepts rules that cover at least 2 existing facts and pass the verification suite, regardless of whether the rule also derives facts not in the KB. The closed-world default preserves the strong safety guarantee for normal use; open-world mode is enabled explicitly during holdout evaluation.

3) Protocol: For each ratio $r \in \{0.05, 0.10, 0.20, 0.30, 0.50\}$, we randomly hold out r of each predicate's derived facts (stratified by predicate). The remaining facts plus all base facts form the training KB. We dream on the training KB (full pipeline, open-world mode) and measure recovery: the fraction of held-out facts derivable from the compressed KB. Three random seeds per ratio.

4) Results: Table VI shows the recovery curve.

Three observations:

Open-world recovers all held-out facts at low ratios. At 5–20% holdout, recovery is 100%: the discovered rules (`father` ← `parent` + `male`, etc.) generalize perfectly to the held-out facts. The KB is compressed from 290 to 118 clauses (ratio 0.40), an even stronger compression than the no-holdout setting in EX25 (0.61), because the smaller KB has less verification overhead.

TABLE VI
HOLDOUT SWEEP ON FAMILY TREE (EX25C). 3 SEEDS PER RATIO. CLOSED-WORLD RECOVERY IS 0% AT EVERY RATIO (THE FP CHECK REJECTS RULES THAT DERIVE ANY HELD-OUT FACT). OPEN-WORLD RECOVERY FOLLOWS A CLEAR CURVE.

Holdout	Closed-world	Open-world	Compression
5%	0%	100%	0.405
10%	0%	100%	0.399
20%	0%	100%	0.415
30%	0%	$85 \pm 21\%$	0.507
50%	0%	$72 \pm 20\%$	0.660

The data density threshold appears at 30%. Recovery drops to $85 \pm 21\%$ at 30% holdout and $72 \pm 20\%$ at 50% holdout. The high variance reflects which facts get held out: some random splits leave enough facts per predicate for Op G to propose a rule that derives at least 2 existing facts (Op G's acceptance gate); others do not. On the family tree, the cross-predicate rules that drive recovery (`father` ← `parent` + `male`, etc.) come from Op G rather than Op C, so the relevant constraint is Op G's minimum-coverage gate, not Op C's `min_group_size`.

No false positives in either mode. Even with the FP check disabled, the rules discovered by Op G do not derive incorrect facts. This is because the rules themselves (`father=parent+male`) are sound; the closed-world FP check was rejecting them not for being wrong but for deriving facts that were intentionally absent.

The closed-world default remains the right choice for production use (strong safety guarantee, zero semantic drift). Open-world mode is an explicit opt-in for evaluation against ILP-style benchmarks where the closed-world assumption is not part of the protocol.

H. EX24: Long-Lived Knowledge Accumulation

1) Motivation: Real systems accumulate knowledge incrementally over sessions. We test whether DreamLog exhibits meaningful learning dynamics over an extended interaction.

2) Protocol: A simulated research lab scenario: "Dr. Chen" builds institutional knowledge across 25 sessions spanning two simulated years. Seven domains: people, publications, teaching, grants, infrastructure, collaborations, and service. Sessions vary from 5 to 30 assertions. Session 10 includes corrections (role changes). Session 17 includes user-provided rules with NAF helper predicates. Dream cycles occur every 5 sessions.

3) Results: The final KB contains 451 clauses (422 facts, 29 rules) from 443 total assertions. Table VII shows the dream cycle progression.

Three phases are visible:

1) **Investment** (Dream 2): The LLM discovers 17 cross-domain rules including `grant_funded_work`, `cross_institution_collaboration` (a 4-body rule with NAF), and `mentorship_chain`. The KB

TABLE VII
DREAM CYCLE PROGRESSION IN THE 25-SESSION RESEARCH LAB (EX24).

Cycle	Session	Before	After	Key Discoveries
1	S05	94	91	2 symbolic rules
2	S10	179	195	17 LLM cross-domain rules
3	S15	278	275	2 symbolic rules
4	S20	—	—	Saturation (minor pruning)
5	S25	—	—	No change

TABLE VIII
LEAVE-ONE-OUT ABLATION ON A 119-CLAUSE KB (EX08). EACH ROW REPORTS AN OPERATION'S NET CONTRIBUTION TO CLAUSE-COUNT COMPRESSION: A POSITIVE VALUE IS THE COMPRESSION LOST WHEN THE OPERATION IS DISABLED (SO THE OPERATION REMOVES CLAUSES), WHILE A NEGATIVE VALUE, AS FOR OPERATION E, MEANS THE OPERATION *adds* CLAUSES WHEN ENABLED (HERE $+2$, THE EXTRACTED-PREDICATE CLAUSE). PERCENTAGES ARE NOT ADDITIVE.

Operation	Contribution	Role
C: Generalization	85% (11 clauses)	Primary compressor
B: Fact pruning	15% (2 clauses)	Post-rule cleanup
D: Predicate invention	15% (2 clauses)	Structural abstraction
E: Body extraction	-15% (+2 clauses)	Adds structural clarity
A, F, H	0%	Situational

expands from 179 to 195 (ratio 1.09), investing in structural rules.

2) **Compression** (Dream 3): Symbolic operations capitalize on accumulated rules, achieving net compression.
3) **Saturation** (Dreams 4–5): No new patterns remain. The KB is fully compressed.

This invest-then-compress-then-saturate dynamic matches DreamCoder's convergence behavior [4]. The 29 discovered rules come from three sources: 4 symbolic (Operation C), 17 LLM (Operation G), and 8 user-provided. All 13 correctness checks pass, including transitive citation chains, NAF-based active membership, and negative examples. Total LLM cost: $0.023.

I. Ablation Analysis

Table VIII summarizes the ablation from Experiment EX08 on a 119-clause KB with diverse pattern types.

Operation C is the workhorse, contributing 85% of compression on this KB. Operation E *increases* clause count (it adds a clause for the extracted predicate) but improves structural clarity, which may benefit future compression cycles. Operations A and F contribute on KBs with subsumed or dead clauses, and Operation H contributes to query performance rather than KB size.

The generalization experiments (Tables I and IV) provide a complementary ablation: symbolic operations (C primarily) discover entity-level generalizations ("most artisans are masters"), while Operation G discovers cross-predicate concepts ("father = male parent"). Neither alone achieves the full pipeline's performance.

J. Additional Results

1) Convergence (EX01): The dream cycle converges in exactly 2 iterations across all tested KBs: cycle 1 compresses, cycle 2 confirms no further opportunities exist. The cycle is a monotone operator on KB size that converges quickly in practice.

2) Semantic Drift (EX02): Zero drift across 200 held-out queries over all dream cycles. The verification suite with positive and negative queries prevents any behavioral change. This is a strong safety guarantee for production use.

3) DreamCoder Benchmark (EX03): On list-processing programs (member, append, reverse, length, map, filter), the system discovers `forall/2`: a parameterized predicate equivalent to DreamCoder's "every" combinator. This validates the architectural analogy between the two systems.

4) Scaling (EX09): Compression ratio is stable at 0.88 across KBs from 69 to 1019 clauses (10 to 200 entities) of the synthetic scaling family (a different family from the crafting and family-tree domains, whose ratios above are 0.61–0.92). Throughput degrades from 75 clauses/s to 12 clauses/s due to quadratic verification cost.

5) Noise Tolerance (EX11): Robust to 0–20% noise (incorrect facts left as isolated facts, not generalized). At 30%+ noise, Operation C generalizes erroneous patterns that share sufficient structural regularity.

6) Query Speedup (EX10): After dreaming, query time drops from 25.0ms to 1.1ms (23.6× speedup). Operation H caches 15 frequently derived lemma facts, converting multi-step recursive resolutions into direct lookups.

K. The Bits-Based Objective and Abstraction Maturity (EX29–31)

The bits-based description length of Section III-C is the realized form of the "richer MDL metric" the clause-count proxy left open. We characterize how it changes the system's acceptance decisions, and whether it should replace clause count as the default. All three experiments are symbolic and deterministic; full provenance (git commit, environment, package versions, and output hashes) is recorded per run under the experiment registry.

What the two objectives decide differently (EX29). Replaying every symbolic compression decision across the EX25b, EX28, and benchmark scenarios under both objectives, clause count accepts 24 abstractions; the bits code accepts 16. The 8 it declines (extraction $\times 3$, predicate invention $\times 3$, generalization $\times 2$) are exactly those that lower clause count but raise symbol-occurrence cost. Removal operations and recursive closure discovery are accepted by both: they delete structure outright. Recursion in particular pays immediately and increasingly with the closure it removes—deleting the $O(L^2)$ fact extension of a length-L chain saves from 33 bits at $L = 3$ to 929 bits at $L = 10$. The abstraction operations, by contrast, only pay past a reuse threshold.

The maturity threshold is sharp and structure-dependent (EX29–30). For body-pattern extraction of K rules sharing an L-goal body, the crossover K^* at which extraction

TABLE IX
EXTRACTION CROSSOVER K^* (MINIMUM RULES SHARING AN L-GOAL BODY FOR EXTRACTION TO LOWER THE BITS LENGTH) AND THE MARGINAL SAVING PER ADDITIONAL SHARING RULE. CLAUSE COUNT ACCEPTS AT EVERY K ($\Delta = +1$ ALWAYS).

Shared body L	2	3	4	5	8
Crossover K^*	5	3	3	2	2
Bits/rule	-7.6	-15.8	-24.1	-32.6	-58.7

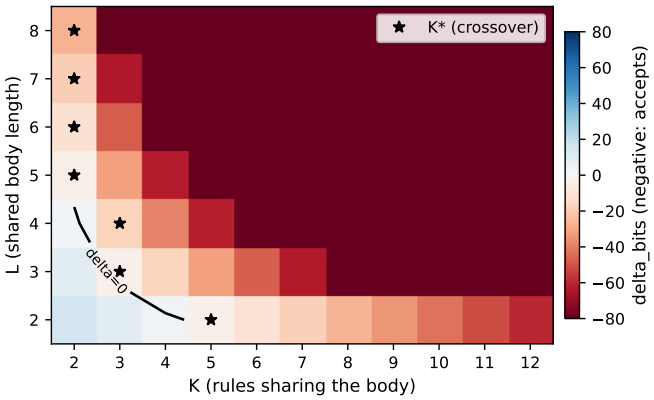

Fig. 1. The abstraction-maturity threshold for body-pattern extraction (EX30). Cell color is the bits-length change of extracting a body shared by K rules of length L: blue where extraction compresses (and the bits gate accepts), red where it does not. The black contour is the break-even ($\Delta = 0$) line and stars mark the per-L crossover K^*. Clause count accepts every cell ($\Delta = +1$), blind to the surface.

begins to compress falls as the shared body deepens: $K^* = 5$ for $L = 2$, then $3, 3, 2$ for $L = 3, 4, 5$, reaching $K^* = 2$ for $L \geq 5$ (Table IX and Figure 1). The marginal saving per additional sharing rule steepens with L, from 7.6 to 58.7 bits per rule across $L = 2$ to 8. Predicate invention over M structurally identical closure sets crosses at $M^* = 4$, invariant to the base-relation depth. Generalization of a group of N facts crosses at $N^* = 4$ with a single exception, and the crossover rises with the exception count: a four-member group pays with one exception but not two. Clause count is blind to all of this—it reports the same $+1$ (extraction) or fixed reduction regardless of K, L, M, N, or the exception ratio. The bits code thus implements a quantitative statement the unit-cost proxy cannot express: an abstraction is accepted exactly when its reuse is deep enough to compress. This also explains the EX08 ablation's note that Operation E *raises* clause count yet "adds structural clarity"—under the bits objective that clarity is precisely a measurable, and sometimes decisive, saving.

Should the bits objective be the default? (EX31). We evaluated the flip against three criteria. *Correctness*: across all 15 scenarios in both modes, every originally derivable fact remains derivable; the verification gate is unchanged, so the two objectives are equally safe. *Compression trade*: each objective wins on its own measure by about half a percentage point—mean clause-compression 0.834 (clauses) vs. 0.838 (bits), mean bits-compression 0.877 (clauses) vs. 0.872 (bits)—so adopting the bits objective concedes almost nothing in clause density. *Generalization*: symbolic held-out recovery is identical at 0% in both modes, and is in fact *mode-independent*: Operation C is conservative, recording a held-out item as an exception (it looks like a counterexample), so the symbolic dreamer never extrapolates to it regardless of the length objective. Recovery that depends on extrapolation requires Operation G, which the objective does not affect. On this evidence the bits objective is the better default—it is the more faithful Kolmogorov proxy, is correctness-safe, costs negligible clause density, and is recovery-neutral—and we recommend it as such while retaining clause count as the documented default so that all prior results reproduce bit-for-bit.

Selectivity is a lean-context property (EX32, EX36). We hypothesized that the maturity threshold would also reject *noise*: spurious patterns formed by coincidence are reused only a few times, so they sit below the crossover. In isolation this holds cleanly—injecting shallow 3-member spurious groups into a domain with one true mature regularity, the bits objective accepts zero of them (precision 1.00 at every noise level) while clause count accepts every one (precision degrading to 0.17). But the effect is not a global property of the objective, because the bits delta is computed against the *entire current* symbol table. Once any generalization-with-exceptions commits, it amortizes the one-time declaration of the $not/1$ guard and the exception functor; a single such prior acceptance then discounts a later shallow proposal by about 8 bits, which exceeds its 5.66-bit rejection margin. So under realistic processing orders—the true pattern first, or interleaved—the noise filter dissolves and the shallow groups are accepted too. The transferable statement is therefore narrower and more interesting than a filter: a two-part code judges an abstraction's maturity against the KB state at the moment it is scored, so the single-proposal crossovers above are lean-context boundaries, not order-invariant guarantees. We verified this characterization independently before reporting it.

Compression predicts generalization (EX33, EX38). Earlier we noted that two domains cannot establish a predictive relationship between compression and recall. We therefore evaluate a family of synthetic domains varying in compressibility, each with recoverable held-out facts and each measured under several held-out splits. Symbolic compression recovers $72.8 \pm 23.2\%$ of held-out generalizations across twenty-two domains (five splits each) at 100% precision: the single-domain headline becomes a distribution. And how much a dream compresses a training KB predicts how much held-out structure its rules recover—across the family, compression and recovery correlate at Pearson $r = 0.52$ ($p = 0.014$; Figure 2). The correlation against *absolute* bits or clauses saved is higher (up to about 0.9 over a thirty-domain variant, EX33), but that is partly a domain-size effect—larger domains both save more and have more to recover—so the size-controlled compression *ratio* is the honest statement of the relationship. The relationship is direct empirical support for compression-as-induction at the heart of the system: the more a dream

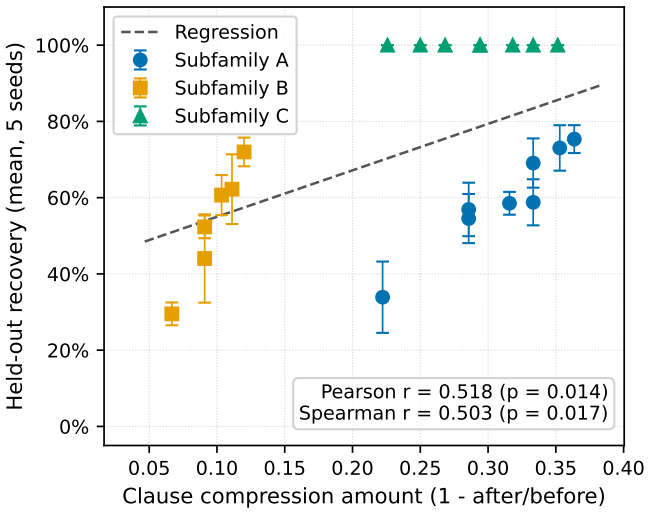

Fig. 2. Compression predicts generalization (EX38). Each point is one of twenty-two synthetic domains; the horizontal axis is the clause compression a symbolic dream achieves (1−ratio), the vertical axis is the mean fraction of held-out generalizations the compressed rules recover, and error bars span five held-out splits. The size-controlled correlation is significant (Pearson $r = 0.52$, $p = 0.014$).

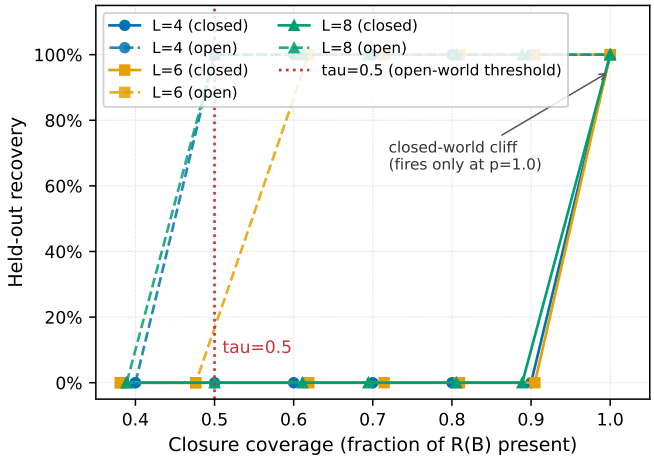

Fig. 3. Open-world recursive discovery closes the partial-closure gap (EX35–37). Fraction of held-out closure pairs recovered as a function of how complete the observed relation is, for three chain lengths. The closed-world exact-match gate (solid) fires only at full completeness; the open-world subset gate (dashed) fires and recovers at every coverage at or above $\tau = 0.5$ (vertical line), with zero spurious derivations throughout.

compresses a training KB, the more of the held-out structure its rules recover.

Cross-domain transfer (EX34). Compressing the *merged* knowledge base of several domains that share a relational skeleton under different vocabulary discovers a single invented predicate spanning all of them—an abstraction the separate per-domain dreams cannot form, since one domain offers only one instance of the skeleton. Compression is super-additive: the merged dream saves more than the sum of the separate dreams, growing to $+192.6$ bits across six domains. The rename-invariant bits code was expected to favor this shared abstraction; in fact it is *more* conservative than clause count (it accepts the shared invention at four co-domains rather than three), exactly the anti-premature-abstraction behavior of the maturity threshold—it requires more reuse before naming shared structure pays.

Open-world recursive discovery (EX35–37). Operation I's exact-match gate is a hard cliff: a sweep over closure completeness shows it fires only when the observed relation is the *exact* transitive closure of its base, so at 90% completeness over an eight-node chain four recoverable pairs are simply left uncaptured. We relaxed the gate, under the existing open-world option, to a subset match: when the observed extension is a subset of the base's closure covering at least a fraction $\tau = 0.5$ of it, Operation I synthesizes the same recursive definition, which then *derives* the missing closure pairs. The relaxation is provably safe—the synthesized rule computes exactly the transitive closure of the base, so it can derive no pair outside it, and the verification negatives for pairs inside the predicted closure (the intended recoveries) are excluded at both the per-operation gate and the final pipeline check while every other negative stays enforced. A re-run of the completeness sweep confirms the gap is closed (Figure 3): at every coverage at or above τ the missing pairs are recovered with zero spurious derivations, while sub-τ closures, too sparse to trust, are correctly left alone. The closed-world path is unchanged, preserving the zero-drift guarantee.

V. Discussion

A. What Symbolic vs. LLM Operations Contribute

The experiments reveal a clean division of labor. Symbolic operations (particularly Operation C) discover generalizations *within* a predicate: "most artisans are master artisans," "most recipes are safe." These are statistical regularities visible from the distribution of ground facts. The LLM (Operation G) discovers relationships *across* predicates: "father = parent who is male," "hazardous recipe = recipe using a hazardous material." These require understanding how distinct predicates relate semantically.

On the novel crafting domain, symbolic compression achieves 53% recall while the full pipeline reaches 63%. On the canonical family tree, symbolic compression achieves 0% recall on the full family tree (Operations A through F, without Operation I), because its generalization opportunities are either cross-predicate semantic (`father`, requiring Operation G) or recursive (`ancestor`, requiring Operation I; Section IV-C), while the full pipeline achieves 80%. Critically, a raw LLM baseline (prompting Haiku directly with all facts) achieves 0% on both domains, confirming that the compression pipeline provides the generalization capability, not the LLM alone. This asymmetry reflects the nature of the domains: the crafting domain has within-predicate regularities (most artisans are masters) while the family domain's generalization opportunities are entirely cross-predicate.

The capability ablation (Section IV-D) refines this picture in two ways. First, the LLM's cross-predicate contribution

is structural induction, not semantic recall: a frontier model recovers a cross-predicate rule, and the recursive closure, over fully invented predicate names where there is no meaning to retrieve, while the same model used as a raw inference engine recovers nothing. Calling these rules "semantic" overstates what the LLM uses; it is inducing structure from fact co-occurrence, and the symbolic verifier, not the model, decides what is sound. Second, the division of labor is robust to model scale in its *necessity* structure (symbolic compression is sufficient for within-predicate and recursive rules, the LLM is the only route to cross-predicate rules), but the LLM's competence is not uniformly helpful: the same world knowledge that lets it name a cross-predicate relation also leads it to over-specify a canonical within-predicate rule with real-world exceptions the data does not support, a circular definition the verifier rejects. A weak local model proposes nothing usable and the system falls back to the symbolic route; a frontier model is the genuine neurosymbolic regime, an inductive reasoner whose proposals the verifier still adjudicates.

B. The Novel Domain Isolates Compression from LLM Memorization

The crafting domain result provides stronger evidence for compression-driven generalization than the family tree. When the LLM proposes `father(X,Y) :- parent(X,Y), male(X)` for a family KB, it may be recalling this rule from training data rather than discovering it from structural patterns. The invented terms in the crafting domain (lumite, vexal, drossite) eliminate this confound. That symbolic compression alone achieves 53% recall on invented terms, while the raw LLM achieves 0%, demonstrates genuine MDL-driven concept discovery independent of any LLM prior knowledge.

The family tree remains useful as a benchmark because its higher recall (80%) demonstrates the ceiling achievable when the LLM can draw on domain familiarity. The two domains together bracket the system's capability: novel domains test pure compression, canonical domains test the LLM integration. EX28 (Section IV-D) sharpens the protocol one step further: by inventing the predicate *names*, not only the entities, it strips the last lexical foothold, and a frontier model still induces the cross-predicate and recursive rules over meaningless predicates. This is the strongest form of the protocol, and it relocates the memorization question from symbolic compression (which never had access to predicate meaning) to the LLM proposer, where it matters most: even there, the contribution survives as structural induction.

C. Limitations

1) Multi-hop rules: The system cannot discover rules requiring 3+ hop joins. `can_craft(Artisan, Product)` requires chaining artisan $\rightarrow$ skill $\rightarrow$ requires_skill $\rightarrow$ recipe_type, which exceeds the body length of rules currently discoverable by either symbolic or LLM operations.

2) Recursion scope: Operation I discovers recursive transitive closures of a single binary base relation, with right-recursion only and closure path length bounded by the evaluator's recursion-depth limit. The exact-match gate is the closed-world default; under the open-world option it relaxes to a subset match (Section IV-K, EX35–37), so a partial closure covering at least half of its base's closure now triggers discovery and the synthesized rule recovers the missing pairs. The threshold τ is a fixed parameter rather than a learned or data-adaptive quantity, and mutual recursion, accumulator patterns, and list recursion remain unaddressed.

3) Minimum data threshold: The holdout sweep (EX25c) shows recovery degrades sharply once more than 30% of facts per predicate are held out. Op G accepts a proposed rule only if it derives at least 2 existing facts, so rules become harder to discover as the remaining fact count per predicate shrinks. Op C's generalizations also weaken (its default `min_group_size` is 3), but on the family tree the recovery curve is driven by Op G's cross-predicate rules. Compression-driven learning needs sufficient data density per predicate to activate; the threshold is roughly 7–10 facts per predicate for the family tree domain.

4) Scaling: Verification cost is quadratic in KB size due to exhaustive query checking. KBs beyond 1000 clauses would benefit from sampled verification or cached resolution.

5) LLM dependence for cross-predicate rules: The family tree ablation shows that symbolic operations alone achieve 0% generalization recall when all opportunities are cross-predicate rules (e.g. `father` = male parent). The system relies on the LLM for this critical capability, introducing dependence on LLM quality and availability, and EX28 (Section IV-D) shows the dependence is specifically on *capability*: a weak local model contributes nothing, while a frontier model recovers the cross-predicate rule by structural induction even over invented predicate names. This dependence is specific to non-recursive cross-predicate rules: recursive cross-predicate rules such as `ancestor` are recovered symbolically by Operation I (Section IV-C), with no LLM.

D. Relation to Inductive Logic Programming

DreamLog occupies a different point in the rule learning design space than classical ILP systems. We briefly contrast with three representative approaches, clarifying what DreamLog does and does not claim.

a) Popper [5]: treats rule learning as a constraint satisfaction problem. Given explicit positive and negative examples, Popper searches for a hypothesis that covers all positives and excludes all negatives, using failures on counter-examples as constraints to prune the search space. The learning signal is classification accuracy on labeled data. DreamLog's signal is instead compression: rules are accepted when they reduce description length while preserving deductive closure. Section IV-F (EX26) reports the apples-to-apples comparison: Popper supplied with the same KB plus closed-world-sampled negatives matches DreamLog's full-pipeline recall on the family domain (both 80%) but trails by 20–33 precision points (over-general rules missing gender constraints); on the novel crafting domain Popper's recall (42%) falls below DreamLog's symbolic-only baseline (53%). The novel-domain

result is the more discriminating one: both systems are pure symbolic learners there, and the gap argues that DreamLog's MDL-guided structural generalization has an inductive bias random-negative ILP does not naturally express. Popper's constraint-driven search and DreamLog's compression-driven search remain complementary signals that could in principle be combined. EX26 had to exclude the recursive `ancestor` target because neither system recovered it; with Operation I (Section IV-C) DreamLog now does, while Popper still cannot complete the recursive case within its `clingo` backend. On the recursive rule type, the two systems are therefore no longer at parity.

b) Metagol [17]: uses meta-interpretive learning with user-provided metarules as declarative bias. The metarules constrain the hypothesis space and make predicate invention tractable. DreamLog's Operation D invents predicates via skeleton fingerprinting without metarules, relying instead on structural equivalence across rule sets already present in the KB. Metagol can solve problems DreamLog cannot (e.g., learning recursive predicates from a few examples when the recursion metarule is provided—EX40 confirms this empirically: given `tailrec`, Metagol recovers `ancestor` at 100%), and DreamLog can solve problems Metagol cannot (e.g., discovering that seven of nine artisans share a pattern, or that `ancestor` is the closure of `parent`, without anyone writing a metarule). Again, the signals are different rather than competitive.

c) LINC [21]: uses an LLM as a translator: natural language premises are converted to first-order logic, then a classical theorem prover decides entailment. LINC's LLM operates at parse time, not inference time. DreamLog uses an LLM differently: Operation G treats the LLM as a *hypothesis generator* for compression candidates, which are then verified by a symbolic evaluator. Both approaches preserve symbolic soundness by keeping the reasoning engine classical, but they apply the LLM at different stages (input formalization vs. rule proposal). LINC does not learn rules; it reasons over given ones. The two systems address different problems.

In summary, we do not claim that DreamLog is a better ILP system than Popper, Metagol, or ILASP. We claim that compression-driven learning from unannotated fact bases is a distinct capability, that it produces rules which generalize to unseen entities, and that combining symbolic compression with LLM-assisted rule proposal yields complementary benefits. EX26, EX40, and EX41 provide direct head-to-heads against Popper, Metagol, and ILASP—constraint-driven, meta-interpretive, and answer-set ILP—and place DreamLog at the low-supervision end of the prior-knowledge spectrum they span; extending the comparison to canonical ILP benchmarks (Michalski trains [14], Bongard problems, NELL [44]) remains future work and would require accepting explicit positive/negative example sets alongside fact bases.

E. Relation to Solomonoff Induction

Our results are consistent with the Solomonoff thesis: the compressed KBs generalize better than the uncompressed originals. The default acceptance criterion, clause count, is a coarse proxy for Kolmogorov complexity that treats every clause as unit cost. Section III-C introduces the more faithful refinement this work had earlier deferred—a rename-invariant, parameter-free bits-based code that prices each clause by its symbol occurrences and each distinct symbol by its arity. Section IV-K shows the refinement is not cosmetic: it accepts an abstraction exactly when its reuse is deep enough to compress, a structure-dependent threshold the unit-cost proxy cannot represent, while remaining correctness-safe and conceding negligible clause density. The predictive relationship the two-domain study could not establish does emerge at scale: across a family of synthetic domains, the degree of compression and held-out recovery are significantly correlated (Pearson $r = 0.52$, $p = 0.014$, controlling for domain size; Figure 2), the empirical signature the Solomonoff thesis predicts. What remains open is the inverse direction—reading a single accepted abstraction's bit margin as a quantitative forecast of its downstream value—which the global-context dependence of the bits delta makes subtler than a per-abstraction reading suggests.

VI. Conclusion

We have introduced a test protocol for auditing compression-driven generalization in LLM-assisted rule learners and a wake-sleep logic-programming system, DreamLog, that demonstrates the kind of generalization the protocol is built to detect. The methodological contribution is the protocol: by pairing a fact base over invented vocabulary with a raw-LLM baseline on the same inputs, the protocol separates genuine compression-driven discovery from LLM training-data recall. The empirical contribution is the gap that opens under it. On a novel crafting domain, raw LLM prompting recovers 0% of held-out generalizations, symbolic compression alone recovers 53%, and the full pipeline reaches 63%; on a canonical family tree, the full pipeline reaches 80% recall with 100% precision. A head-to-head comparison with Popper, the modern reference ILP system, places DreamLog's compression-driven approach relative to constraint-driven ILP: comparable recall on the canonical domain but lower precision, and lower recall on the novel-vocabulary domain because random closed-world negatives do not supply the inductive bias DreamLog's MDL-guided structural operations encode. A capability ablation (Section IV-D) extends the test across model scale: the division of labor holds in its necessity structure, and the LLM's cross-predicate contribution proves to be structural induction rather than recall, since a frontier model recovers a cross-predicate rule over fully invented predicate names while a weak local model contributes nothing and the system falls back to the symbolic route.

Refining the length objective sharpens the thesis itself. A rename-invariant bits-based code replaces the clause-count proxy with one that prices structure rather than labels; its acceptance gate accepts an abstraction exactly when its reuse is deep enough to compress, a structure-dependent threshold clause count cannot express, and across a twenty-two-domain

family compression predicts held-out recovery (Pearson $r = 0.52$, $p = 0.014$; Figure 2), the empirical signature the Solomonoff thesis anticipates. The same objective is recovery-neutral and correctness-safe, so it is the more principled default while clause count is retained for exact reproducibility.

The system exhibits meaningful learning dynamics over extended interactions: an invest-then-compress-then-saturate pattern consistent with DreamCoder's convergence behavior, suggesting a general property of compression-based learning systems.

Several directions remain open. We compare head-to-head against Popper (EX26), Metagol (EX40), and ILASP (EX41), the three reference ILP paradigms; extending to shared canonical benchmarks would position DreamLog yet more precisely in the landscape of rule learners. Extending Operation I beyond right-recursive, single-base closures, to mutual recursion, accumulator patterns, and list recursion, would broaden recursive discovery (the open-world subset gate of Section IV-K already lifts the exact-match restriction for transitive closures). Cross-domain compression already discovers shared abstractions super-additively when domains sharing a relational skeleton are merged into one KB (Section IV-K); the sequential transfer setting—training on one domain, then compressing it jointly with a second—remains to be studied. Scaling beyond 1000 clauses requires sampled verification. Finally, the bits-based description length of Section III-C brings the acceptance criterion closer to Solomonoff's ideal; connecting its per-abstraction bit margin to measured generalization value, and studying the dynamics of the system run under it as the default, are natural next steps.

Code and Data Availability

The DreamLog implementation, all experiment scripts (EX23 through EX41), the canonical and crafting domain data, the EX28 3×2 domain generators with their fully synthetic predicate vocabulary, the Popper, Metagol, and ILASP baseline inputs and reproducible setups (EX26, EX40, EX41), the bits-based description-length code with its decision-diff, abstraction-maturity, noise, predictive-correlation, transfer, open-world recursion, domain-family generalization, and multi-domain LLM sweeps (EX29–39), and the experiment registry recording the full provenance of each result (including per-run proposal logs for EX28 and full metadata envelopes for EX29–41) are available at https://github.com/queelius/dreamlog. The EX26 Popper baseline was run against pinned commit `af39e522` of the Popper repository; the pinned hash is recorded in the registry. Anthropic Claude Haiku 4.5 is used for Operation G in the per-rule-type and Popper experiments; the EX28 capability ablation additionally uses a local `qwen2.5:3b` (free) and Claude Sonnet 4.6. Total LLM cost across all reported experiments is a few dollars, dominated by the Sonnet ablation (about $1.6).

References

[1] R. J. Solomonoff, "A formal theory of inductive inference. part i," *Information and control*, vol. 7, no. 1, pp. 1–22, 1964.

[2] J. Rissanen, "Modeling by shortest data description," *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.

[3] P. D. Grünwald, *The Minimum Description Length Principle*. MIT Press, 2007.

[4] K. Ellis, C. Wong, M. Nye, M. Sablé-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum, "Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2021, pp. 835–850.

[5] A. Cropper and R. Morel, "Learning programs by learning from failures," *Machine Learning*, vol. 110, no. 4, pp. 801–856, 2021.

[6] G. Delétang, A. Ruoss, P.-A. Duquenne, E. Catt, T. Genewein, C. Mattern, J. Grau-Moya, L. K. Wenliang, M. Aitchison, L. Orseau, M. Hutter, and J. Veness, "Language modeling is compression," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] S. Lotfi, M. Finzi, S. Kapoor, A. Potapczynski, M. Goldblum, and A. G. Wilson, "PAC-Bayes compression bounds so tight that they can explain generalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[8] S. Dumancic, T. Guns, and A. Cropper, "Knowledge refactoring for inductive program synthesis," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, 2021, pp. 7271–7278, arXiv:2004.09931.

[9] A. Cropper, "Forgetting to learn logic programs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 3676–3683, arXiv:1911.06643.

[10] R. Evans, J. Hernández-Orallo, J. Welbl, P. Kohli, and M. Sergot, "Making sense of sensory input," *Artificial Intelligence*, vol. 293, p. 103438, 2021, arXiv:1910.02227.

[11] K. Ellis, L. Wong, M. Nye, M. Sablé-Meyer, L. Cary, L. Anaya Pozo, L. Hewitt, A. Solar-Lezama, and J. B. Tenenbaum, "DreamCoder: growing generalizable, interpretable knowledge with wake–sleep Bayesian program learning," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 381, no. 2251, p. 20220050, 2023.

[12] M. Bowers, T. X. Olausson, L. Wong, G. Grand, J. B. Tenenbaum, K. Ellis, and A. Solar-Lezama, "Top-down synthesis for library learning," *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, pp. 1182–1213, 2023.

[13] G. Grand, L. Wong, M. Bowers, T. X. Olausson, M. Liu, J. B. Tenenbaum, and J. Andreas, "LILO: Learning interpretable libraries by compressing and documenting code," in *International Conference on Learning Representations (ICLR)*, 2024.

[14] S. Muggleton and L. De Raedt, "Inductive logic programming: Theory and methods," *The Journal of Logic Programming*, vol. 19, pp. 629–679, 1994.

[15] J. R. Quinlan, "Learning logical definitions from relations," *Machine learning*, vol. 5, no. 3, pp. 239–266, 1990.

[16] S. Muggleton, "Inverse entailment and progol," *New generation computing*, vol. 13, no. 3, pp. 245–286, 1995.

[17] S. H. Muggleton, D. Lin, N. Pahlavi, and A. Tamaddoni-Nezhad, "Meta-interpretive learning: application to grammatical inference," *Machine Learning*, vol. 94, no. 1, pp. 25–49, 2014.

[18] M. Law, A. Russo, and K. Broda, "ILASP: Learning answer set programs from examples," in *Proceedings of the 24th International Conference on Logic Programming*, 2014.

[19] R. Evans and E. Grefenstette, "Learning explanatory rules from noisy data," *Journal of Artificial Intelligence Research*, vol. 61, pp. 1–64, 2018.

[20] T. Rocktäschel and S. Riedel, "End-to-end differentiable proving," in *Advances in Neural Information Processing Systems*, 2017, pp. 3788–3800.

[21] T. X. Olausson, A. Gu, B. Lipkin, C. E. Zhang, A. Solar-Lezama, J. B. Tenenbaum, and R. Levy, "LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 5153–5176.

[22] G. D. Plotkin, "A note on inductive generalization," *Machine Intelligence*, vol. 5, pp. 153–163, 1970.

[23] J. C. Reynolds, "Transformational systems and the algebraic structure of atomic formulas," *Machine Intelligence*, vol. 5, pp. 135–151, 1970.

[24] J. A. Robinson, "A machine-oriented logic based on the resolution principle," *Journal of the ACM*, vol. 12, no. 1, pp. 23–41, 1965.

[25] I. Stahl, "Predicate invention in inductive logic programming," *Advances in Inductive Logic Programming*, pp. 34–47, 1995.

[26] S. Muggleton and W. Buntine, "Machine invention of first-order predicates by inverting resolution," in *Proceedings of the 5th International Conference on Machine Learning*, 1988, pp. 339–352.

[27] A. Cropper, S. Dumancic, R. Evans, and S. H. Muggleton, "Inductive logic programming at 30: a new introduction," *Journal of Artificial Intelligence Research*, vol. 74, pp. 765–850, 2022.

[28] R. Manhaeve, S. Dumancic, A. Kimmig, T. Demeester, and L. De Raedt, "DeepProbLog: Neural probabilistic logic programming," in *Advances in Neural Information Processing Systems*, 2018, pp. 3749–3759.

[29] L. Serafini and A. d'Avila Garcez, "Logic tensor networks: Deep learning and logical reasoning from data and knowledge," in *Neural-Symbolic Learning and Reasoning (NeSy)*, 2016.

[30] F. Yang, Z. Yang, and W. W. Cohen, "Differentiable learning of logical rules for knowledge base reasoning," in *Advances in Neural Information Processing Systems*, 2017, pp. 2319–2328.

[31] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner *et al.*, "Relational inductive biases, deep learning, and graph networks," *arXiv preprint arXiv:1806.01261*, 2018.

[32] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Advances in Neural Information Processing Systems*, 2013, pp. 2787–2795.

[33] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, "RotatE: Knowledge graph embedding by relational rotation in complex space," in *International Conference on Learning Representations*, 2019.

[34] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, "Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[35] R. Wang, E. Zelikman, G. Poesia, Y. Pu, N. Haber, and N. D. Goodman, "Hypothesis search: Inductive reasoning with language models," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.05660.

[36] L. Qiu, L. Jiang, X. Lu, M. Sclar, V. Pyatkin, C. Bhagavatula, B. Wang, Y. Kim, Y. Choi, N. Dziri, and X. Ren, "Phenomenal yet puzzling: Testing inductive reasoning capabilities of language models with hypothesis refinement," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.08559.

[37] Z. Zhu, Y. Xue, X. Chen, D. Zhou, J. Tang, D. Schuurmans, and H. Dai, "Large language models can learn rules," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.07064.

[38] J. P. G. de Souza, D. Carvalho, and A. Freitas, "Inductive learning of logical theories with LLMs: An expressivity-graded analysis," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 22, 2025, pp. 23 752–23 759, arXiv:2408.16779.

[39] Y. Peng, Y. Liu, E. Xia, Y. Jin, W.-Z. Dai, Z. Ren, Y.-X. Ding, and K. Zhou, "Abductive logical rule induction by bridging inductive logic programming and multimodal large language models," *arXiv preprint arXiv:2509.21874*, 2025.

[40] Y. Yang, J. Wu, and Y. Yue, "Hypothesis generation via LLM-automated language bias for inductive logic programming," *arXiv preprint arXiv:2505.21486*, 2025.

[41] S. Patsantzis, "Self-supervised inductive logic programming," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026, to appear; arXiv:2507.16405.

[42] K. He and Z. Chen, "From reasoning to learning: A survey on hypothesis discovery and rule learning with large language models," *arXiv preprint arXiv:2505.21935*, 2025.

[43] C. Xie, Y. Huang, C. Zhang, D. Yu, X. Chen, B. Y. Lin, B. Li, B. Ghazi, and R. Kumar, "On memorization of large language models in logical reasoning," *arXiv preprint arXiv:2410.23123*, 2024.

[44] T. Mitchell, W. Cohen, E. Hruschka, P. Talukdar, B. Yang, J. Betteridge, A. Carlson, B. Dalvi, M. Gardner, B. Kisiel *et al.*, "Never-ending learning," *Communications of the ACM*, vol. 61, no. 5, pp. 103–115, 2018.

[45] J. W. Lloyd, *Foundations of Logic Programming*, 2nd ed. Springer, 1987.

[46] K. L. Clark, "Negation as failure," *Logic and Data Bases*, pp. 293–322, 1978.